

Supplementary Information for
Dynamics of Batch Protein Precipitation

May 9, 2025

Amirkiarash Ehtesham¹, Abhishek Sivaram¹, Sara Danielle Siegel²,
John van Zanten², and Seyed Soheil Mansouri¹

¹Department of Chemical and Biochemical Engineering,
Technical University of Denmark, Kgs. Lyngby, Denmark

²Golden LEAF Biomanufacturing Training and Education Center (BTEC),
North Carolina State University, Raleigh, NC, USA

Contents

1	Experimental setup:	3
2	Phenomenology of population balance model:	3
2.1	PBM constant parameters or system parameters	4
2.2	Solving ODE system:	4
3	Differential evolution optimizer internal variables:	4
4	PBM optimized results and measurements:	5
5	PBM surrogate prediction results:	21
6	Neural network internal variables and structure:	36
7	Neural network training and validation	37

1 Experimental setup:

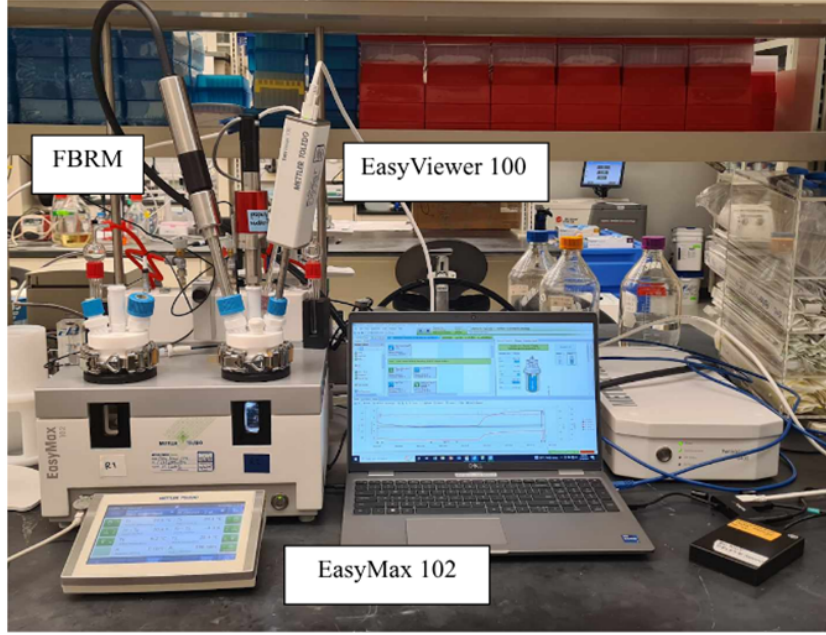


Figure 1: Experimental setup of the EasyMax 102 instrument in the BTEC laboratory with Particletrack G400 (on the left side) and the Easyviewer 100 (on the right side).

2 Phenomenology of population balance model:

In precipitation and flocculations and other similar unit operations there are several different phenomena that can change and alter the particle size distribution. The birth and death terms of these phenomena are summarized in the following table and they are put together to make equation 1.

$$\frac{\partial N_i(v, t)}{\partial t} = B_i - D_i \quad (1)$$

	Birth rate		Death rate
Nucleation	$B_{i=1} = \alpha$ $B_{i \neq 1} = 0$		$D_i = 0$
Growth	$B_{i=1} = 0$ $B_{i \neq 1} = \beta_{i-1} \cdot \frac{1}{2\Delta L_{i-1}} N_{i-1}$ $B_{i \neq m} = \gamma_{i+1} \cdot \frac{1}{2\Delta L_{i+1}} N_{i+1}$		$D_{i \neq m} \cdot \frac{1}{2\Delta L_i} \cdot N_i$ $D_{i=m} = 0$
Shrinkage	$B_{i=m} = 0$		$D_i = \gamma_i \cdot \frac{1}{2\Delta L_i} \cdot N_i$
Agglomeration	$B_i = \left(\sum_{k=0}^m \sum_{j=k}^m (1 - 0.5 \cdot \delta_{j,k}) \alpha_{j,k} \cdot \beta_{j,k} \cdot \eta_{j,k,i} \cdot N_j \cdot N_k \right)$		$D_i = N_i \cdot \sum_k \alpha_{i,k} \cdot \beta_{i,k} \cdot N_k$
Breakage	$B_i = \left(\sum_j \gamma_{j,i} \text{ modified} \cdot S_j N_j \right)$		$D_i = S_i \cdot N_i$

Table 1: Different mechanisms and phenomena that are present in particulate processes, corresponding discrete birth and death term [1].

The second constraint dictates that for instances in which $i > j$, the fractions must be nullified, as it is inconceivable for a larger particle to emerge from the breakage of a smaller particle.

2.1 PBM constant parameters or system parameters

Name	System parameter	Value	Units
Used reactor volume	V_L	40	mL
Minimum particle length in the system	L_{min}	2	nm
Maximum particle length in the system	L_{max}	1	cm
Primary particle size	L_p	2	nm
Fractal Dimension	D_f	2	N/A
Reactor inner diameter	d	52	mm
Magnetic stirbar diameter	D	25	mm
Magnetic stirbar height	h_1	10	mm
Density of the system	ρ_c	1000	$\frac{\text{kg}}{\text{m}^3}$
Dynamic Viscosity	η	0.8891	mPa·s
Kinematic Viscosity	ν	8.91E-07	m^2s^{-1}

Table 2: System parameters and their respective values and units.

To get an initial Distribution of the system, it is assumed that all the protein particles are primary particles with a size of $2nm$. To get the total particles in the system, It can be calculated from Equation 2. The density of lysozyme is assumed to be $14300 \frac{\text{Kg}}{\text{m}^3}$ ([2]).

$$N_0 = \frac{\text{protein concentration [g/L]} \times V_L [\text{L}] \times 6.022 \times 10^{23}}{1000 \times 14300} \quad (2)$$

2.2 Solving ODE system:

To balance computational efficiency with accuracy, the total number of particle bins was set to 30. This binning choice ensures a sufficient resolution to capture particle distribution dynamics effectively.

The initial condition was constructed by assuming that all particles present, based on the specific protein concentration and reaction volume, act as primary particles in the experiment.

The ODE system was then solved using the `solve_ivp` function from Python’s SciPy library, with the LSODA method chosen due to its robust handling of stiffness, which enhances solution stability and convergence. The absolute tolerance is 1e-3 and relative tolerance is 1e-6.

3 Differential evolution optimizer internal variables:

Table 3 lists the internal variables used in the differential evolution (DE) optimizer in Python. Each variable plays a specific role in how the optimizer searches for the best solution. Below is a detailed explanation of each variable:

- **maxiter:** This sets the maximum number of iterations (or generations) that the optimizer will go through. In this case, it is set to 3, meaning the optimizer will refine the population

Table 3: Internal variables used in the differential evolution optimizer in Python

Optimizer variables	values
maxiter	3
popsiz	40
strategy	'randtobest1bin'
mutation	(0.5, 1.5)
recombination	0,7
seed	np.random.default_rng(seed=7)

three times before stopping. A low value like this is often used for testing or if quick results are needed.

- **popsiz**: This defines the population size, or how many candidate solutions exist in each generation. Here, it is set to 10, meaning there will be 10 possible solutions being evaluated at each step. A larger population can give better results but increases computation time.
- **strategy**: The strategy decides how new candidates are generated. The value 'randtobest1bin' means the optimizer will use a random vector and the best solution so far to create new solutions. The 'bin' part refers to binary crossover, a method for combining solutions.
- **mutation**: This is a range that controls how much variation (or randomness) is introduced into the solutions. The values (0.5, 1.5) indicate that the mutation factor can vary within this range, which helps the optimizer explore the solution space more effectively.
- **recombination**: This determines the likelihood of combining information from different solutions. A value of 0.7 means there is a 70% chance of combining candidate solutions. This balance helps maintain diversity in the population while converging toward a good solution.
- **seed**: The seed is used to initialize the random number generator, ensuring reproducibility of results. In this case, a NumPy random generator (`np.random.default_rng(seed=7)`) is used with a fixed seed value of 7. This makes the optimization process deterministic, meaning the same results will be obtained every time it is run.

By using these settings, the optimizer is configured to quickly test potential solutions while still maintaining some flexibility and randomness to avoid getting stuck in local minima. These values might need adjustment depending on the complexity of the problem and the desired accuracy of the results.

4 PBM optimized results and measurements:

All of the experimental and modeling predictions are given in the following section.

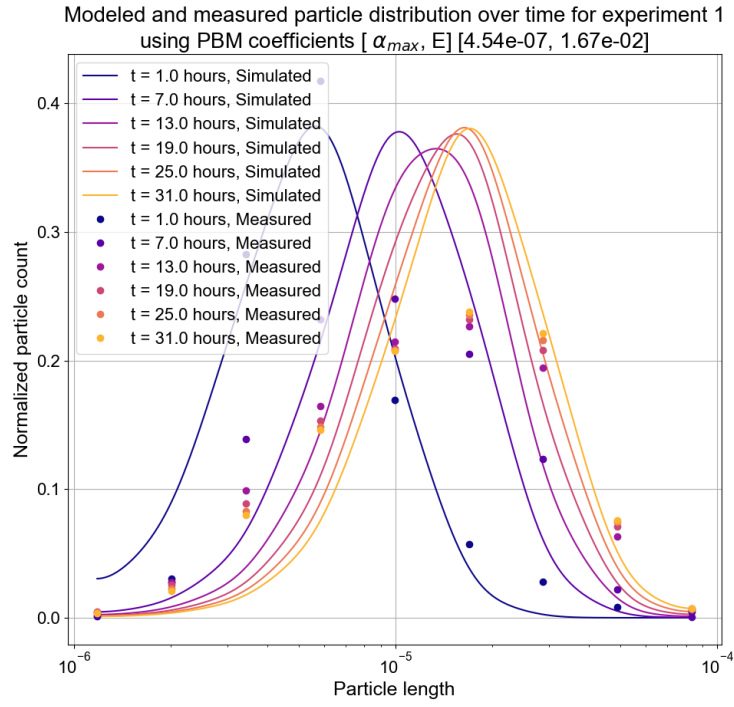


Figure 2: Particle size distributions from PBM model predictions and experimental data for experiment 1

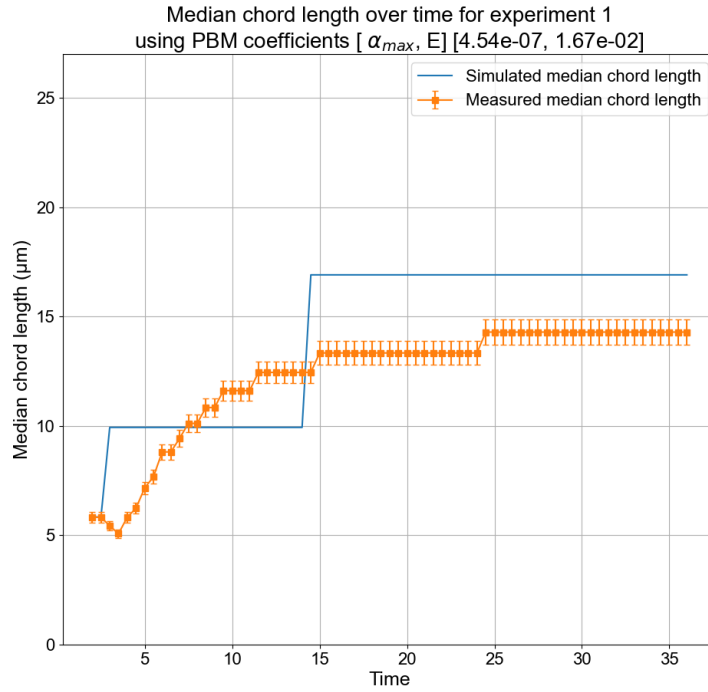


Figure 3: Median of particle size distributions from PBM prediction and experimental data for experiment 1

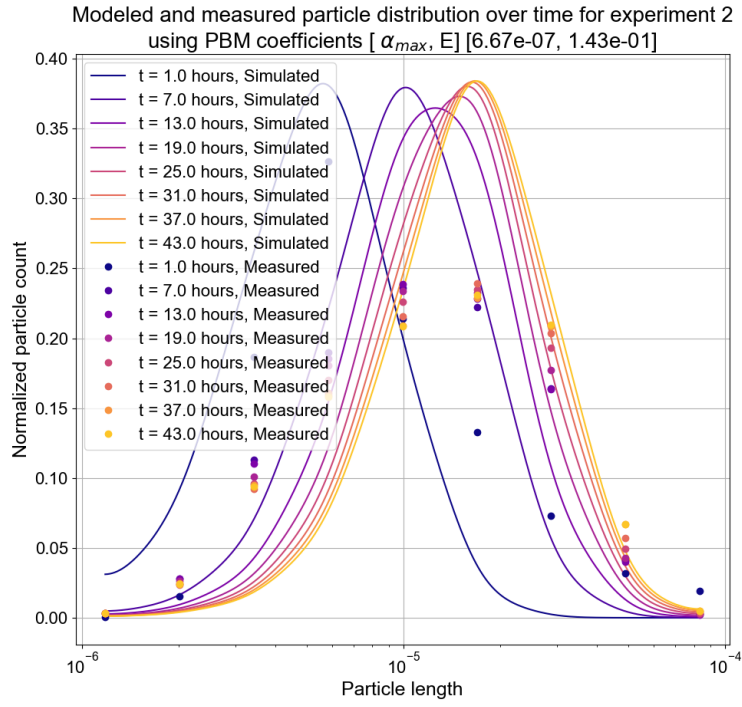


Figure 4: Particle size distributions from PBM model predictions and experimental data for experiment 2

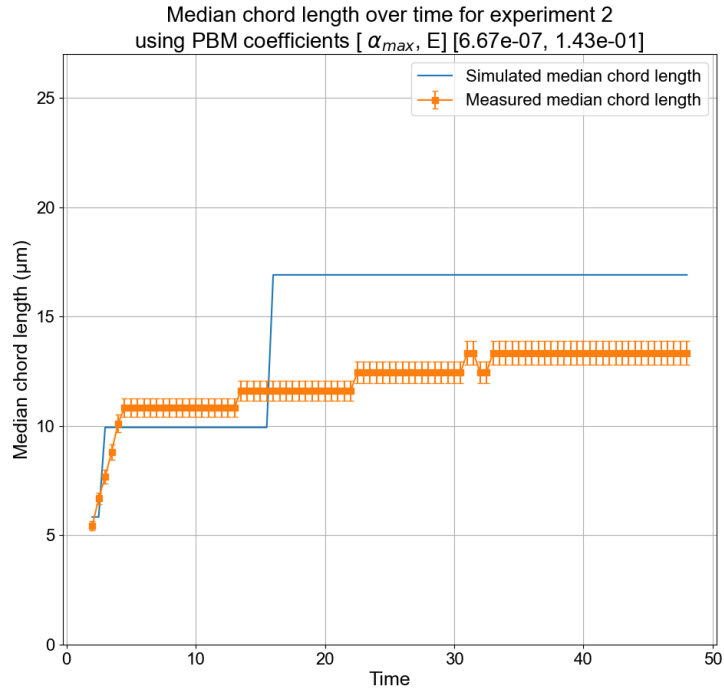


Figure 5: Median of particle size distributions from PBM prediction and experimental data for experiment 2

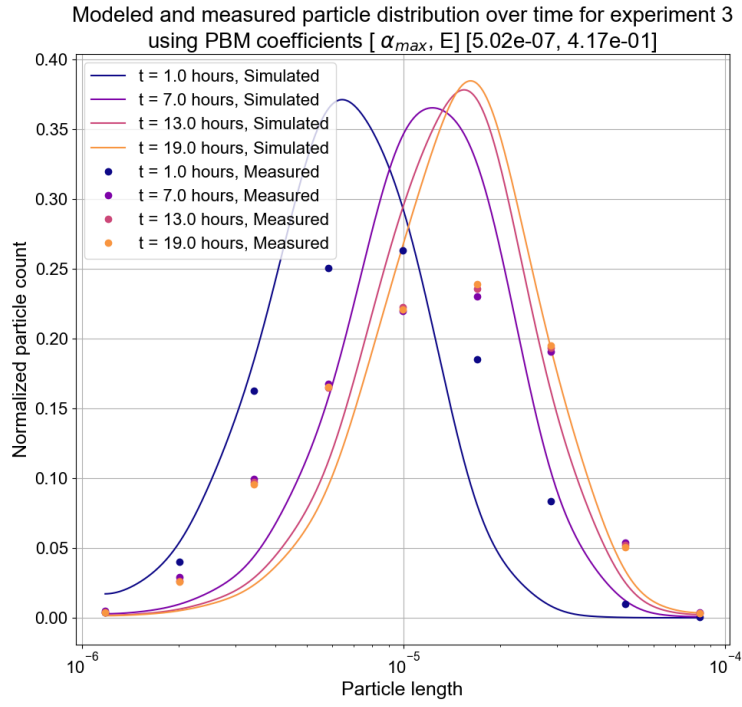


Figure 6: Particle size distributions from PBM model predictions and experimental data for experiment 3

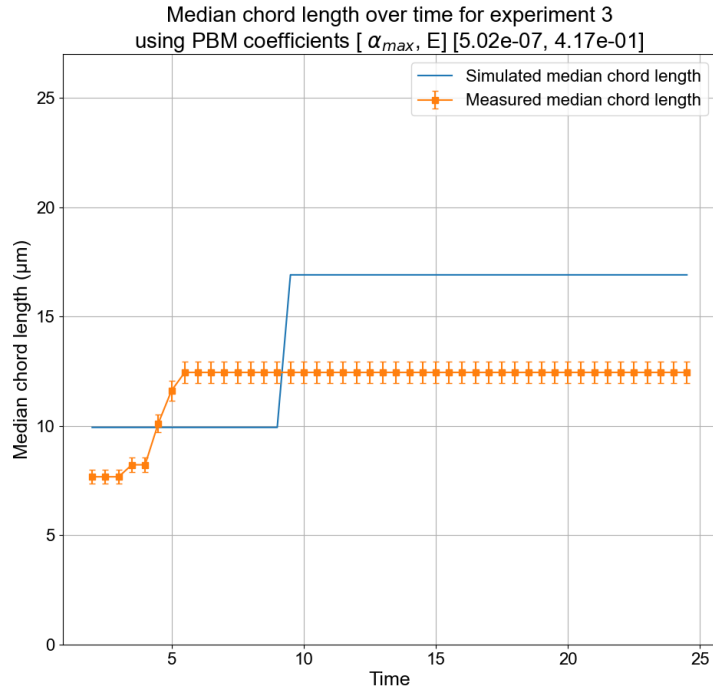


Figure 7: Median of particle size distributions from PBM prediction and experimental data for experiment 3

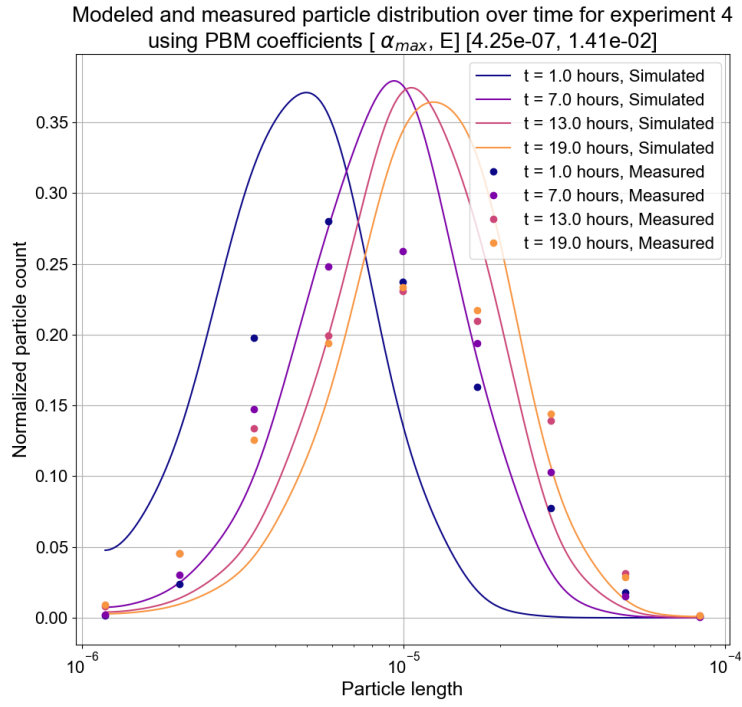


Figure 8: Particle size distributions from PBM model predictions and experimental data for experiment 4

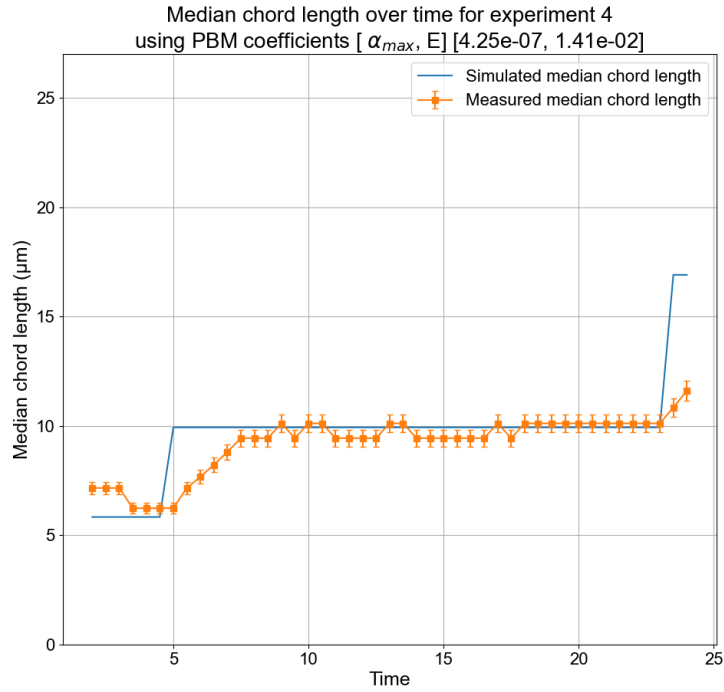


Figure 9: Median of particle size distributions from PBM prediction and experimental data for experiment 4

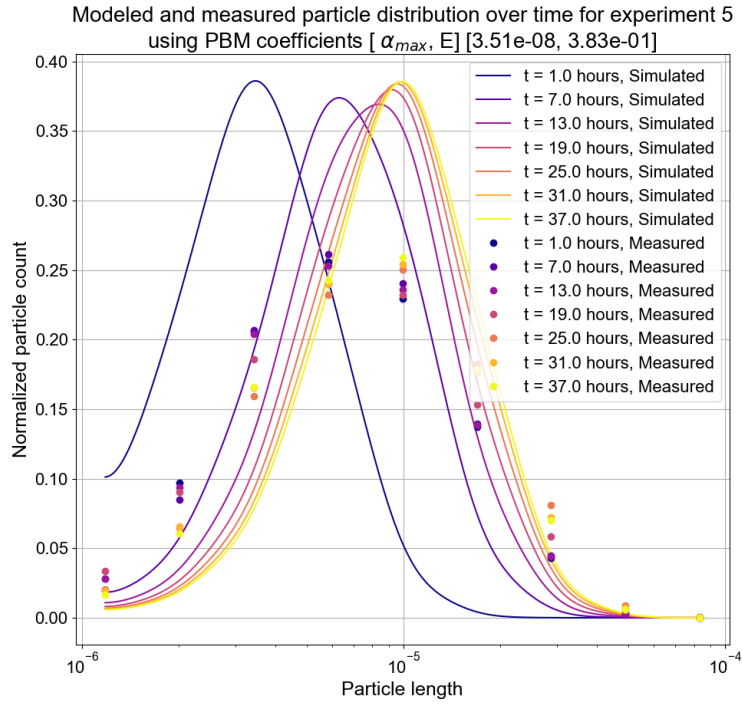


Figure 10: Particle size distributions from PBM model predictions and experimental data for experiment 5

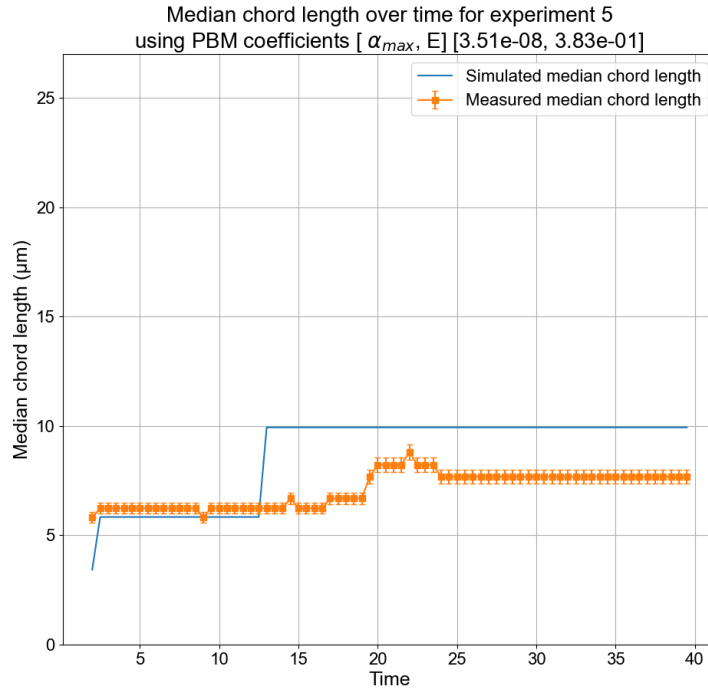


Figure 11: Median of particle size distributions from PBM prediction and experimental data for experiment 5

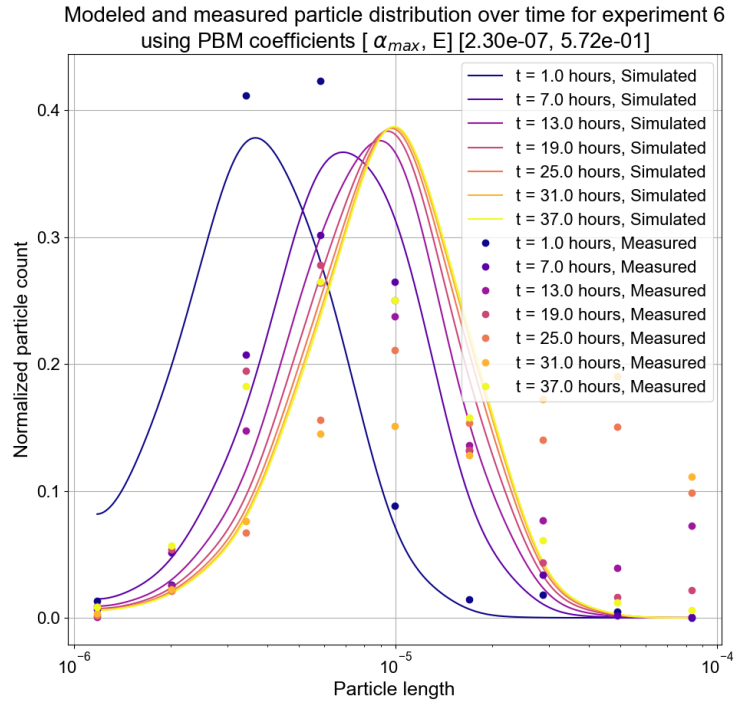


Figure 12: Particle size distributions from PBM model predictions and experimental data for experiment 6

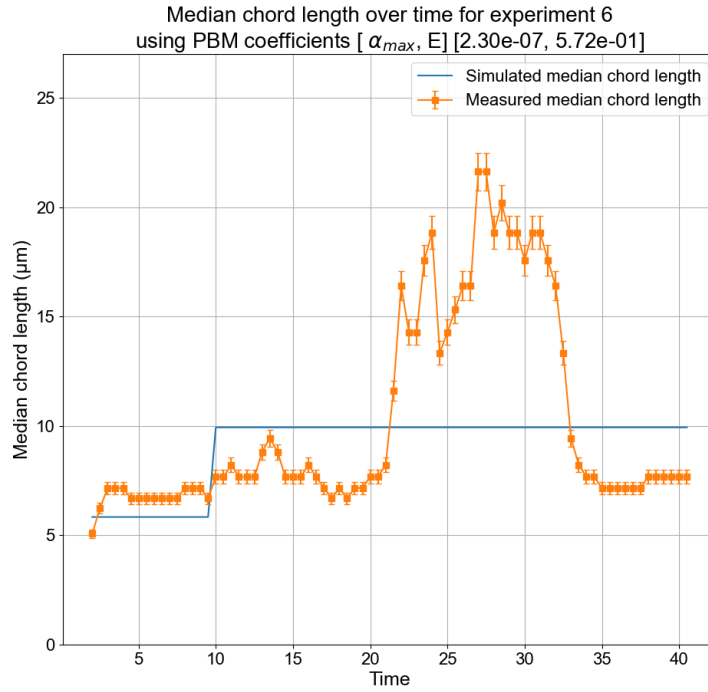


Figure 13: Median of particle size distributions from PBM prediction and experimental data for experiment 6

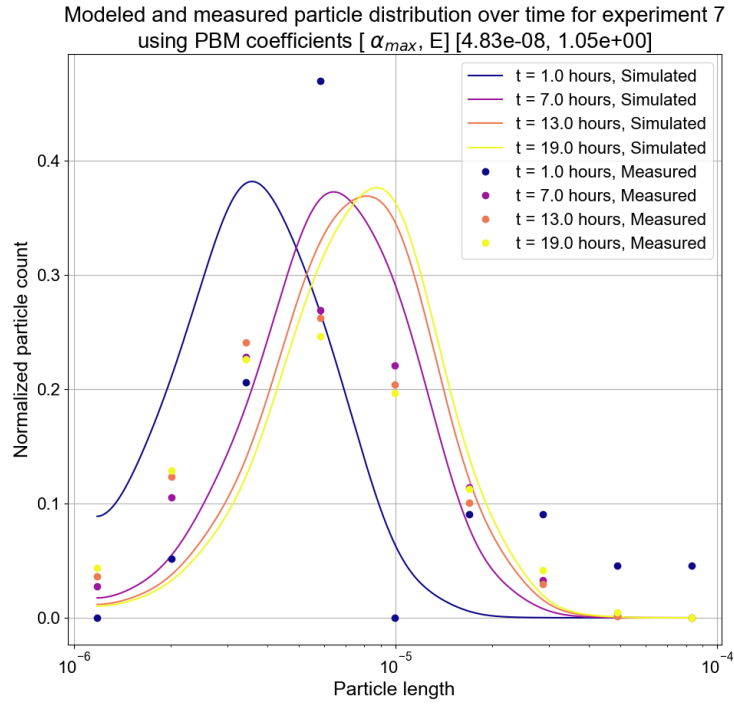


Figure 14: Particle size distributions from PBM model predictions and experimental data for experiment 7

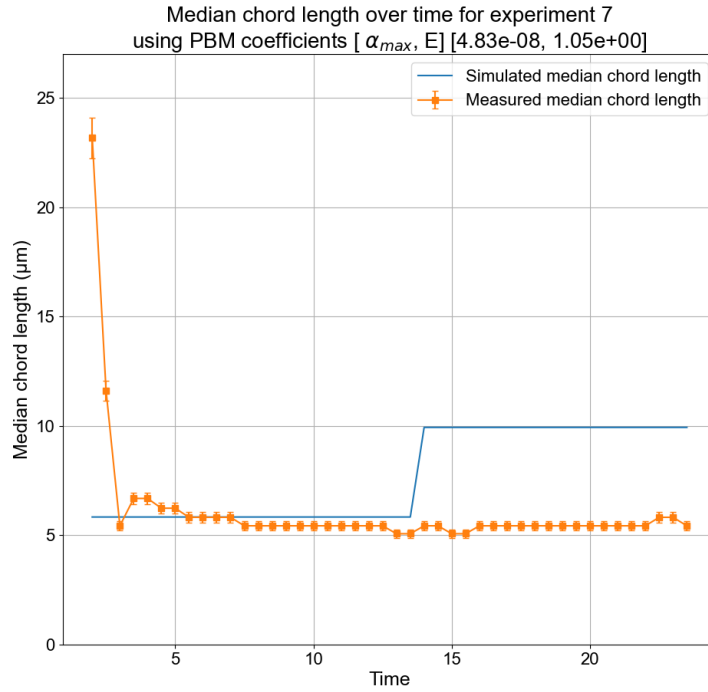


Figure 15: Median of particle size distributions from PBM prediction and experimental data for experiment 7

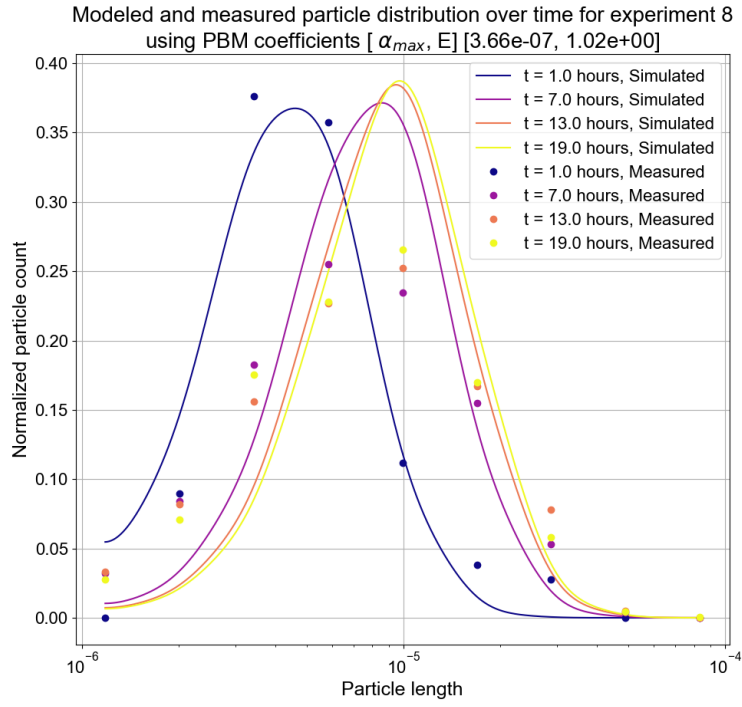


Figure 16: Particle size distributions from PBM model predictions and experimental data for experiment 8

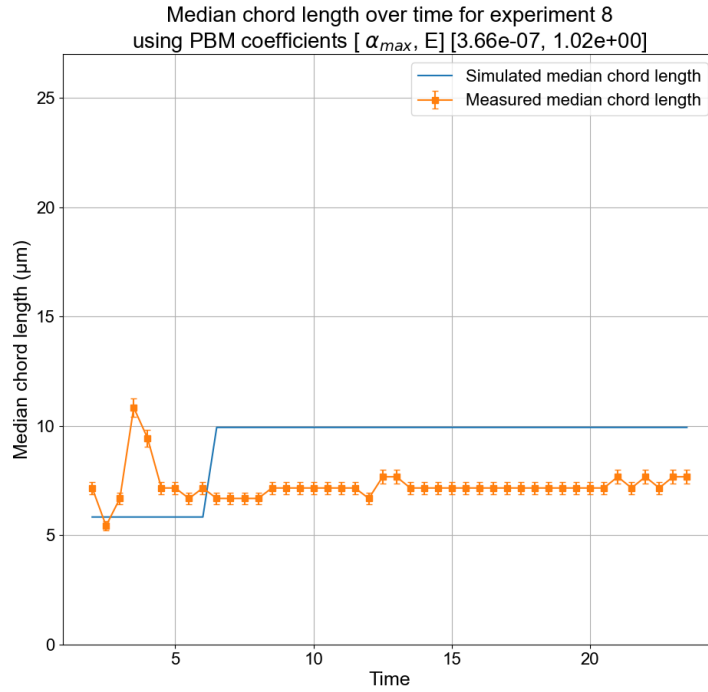


Figure 17: Median of particle size distributions from PBM prediction and experimental data for experiment 8

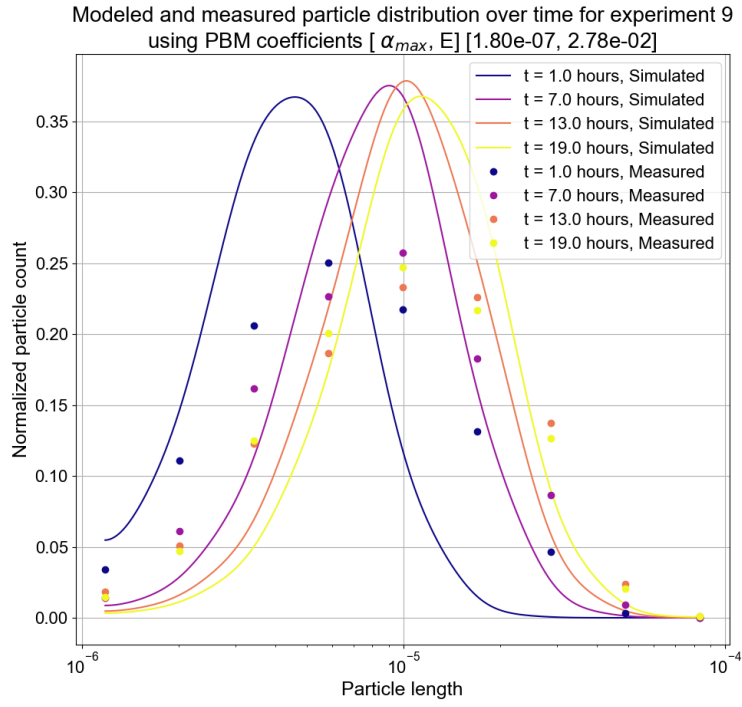


Figure 18: Particle size distributions from PBM model predictions and experimental data for experiment 9

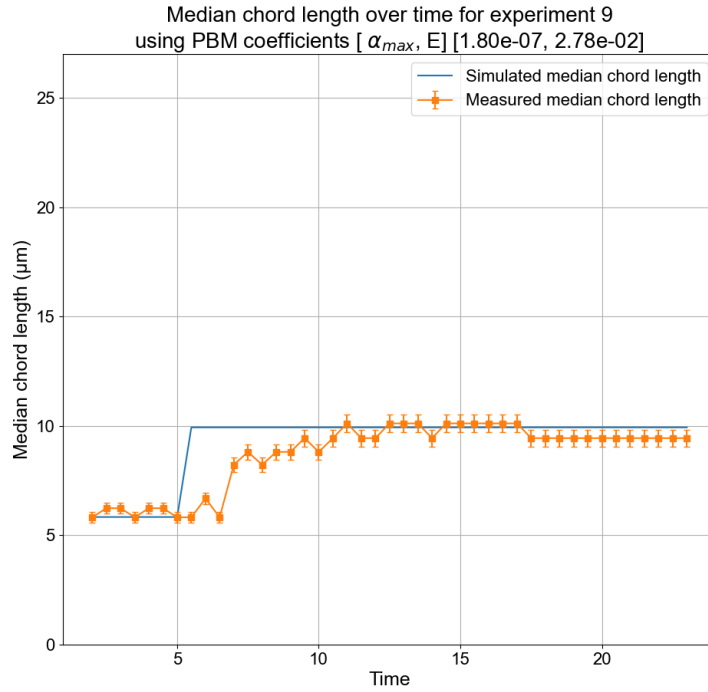


Figure 19: Median of particle size distributions from PBM prediction and experimental data for experiment 9

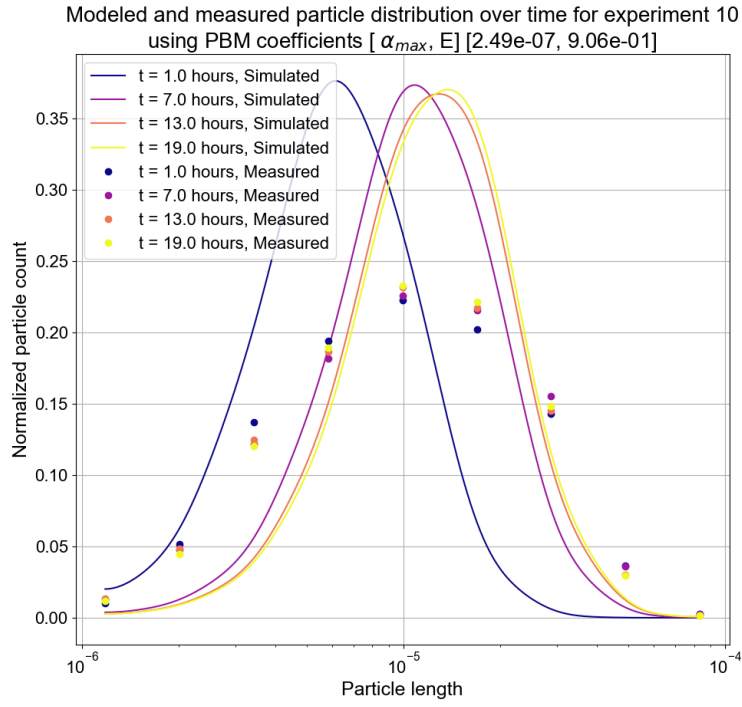


Figure 20: Particle size distributions from PBM model predictions and experimental data for experiment 10

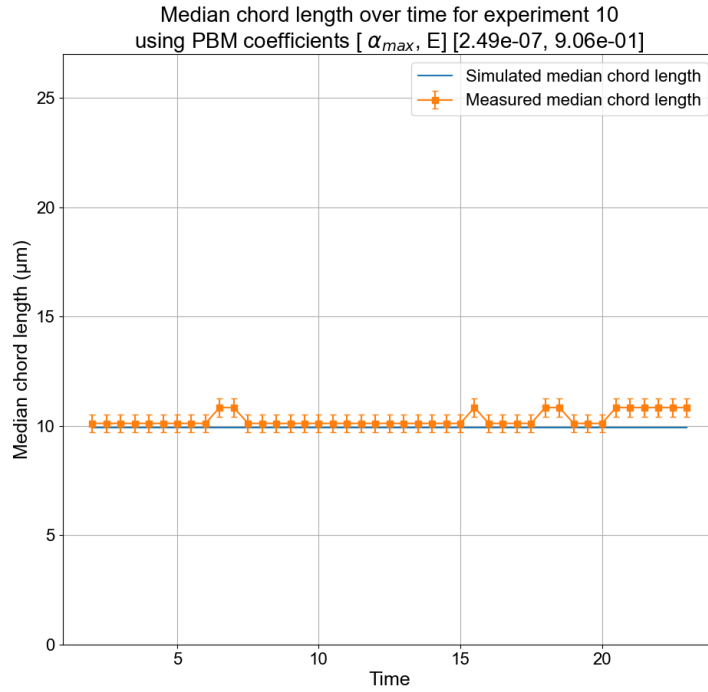


Figure 21: Median of particle size distributions from PBM prediction and experimental data for experiment 10

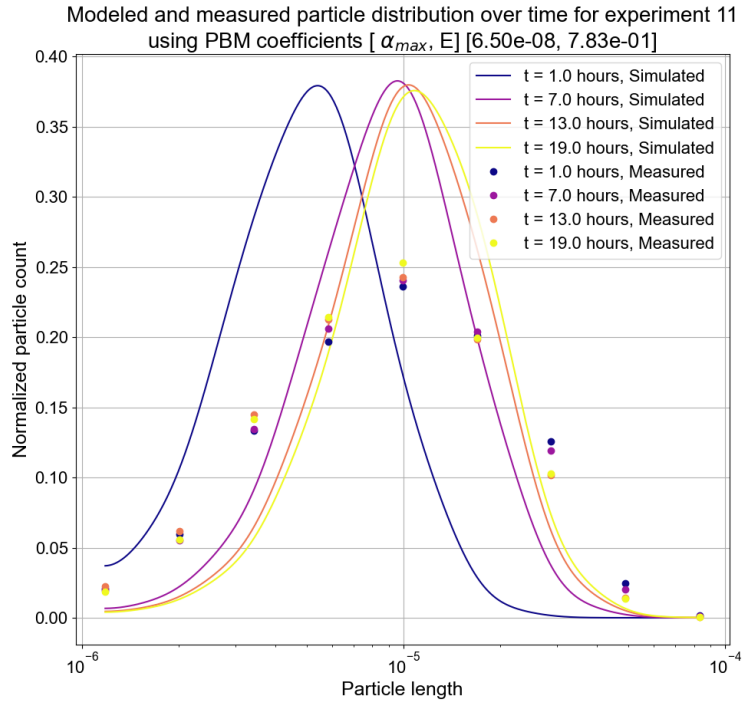


Figure 22: Particle size distributions from PBM model predictions and experimental data for experiment 11

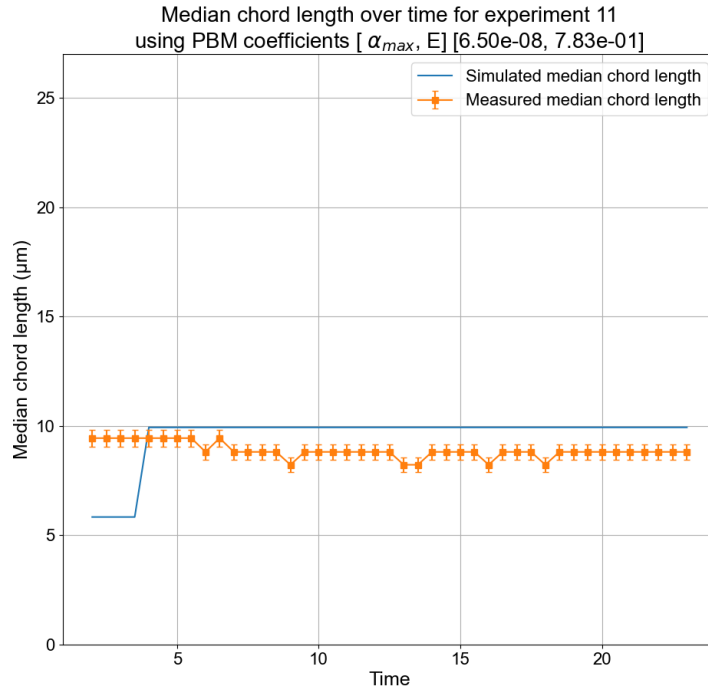


Figure 23: Median of particle size distributions from PBM prediction and experimental data for experiment 11

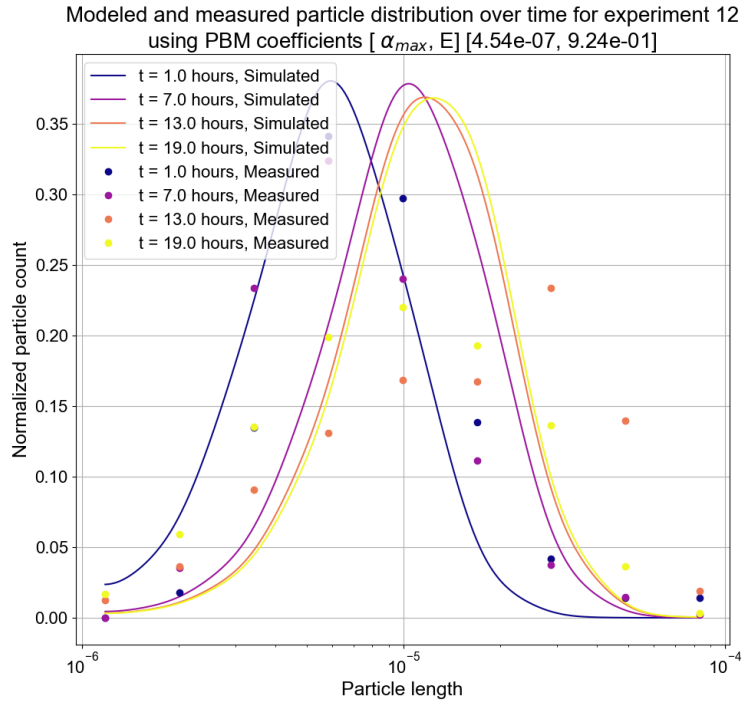


Figure 24: Particle size distributions from PBM model predictions and experimental data for experiment 12

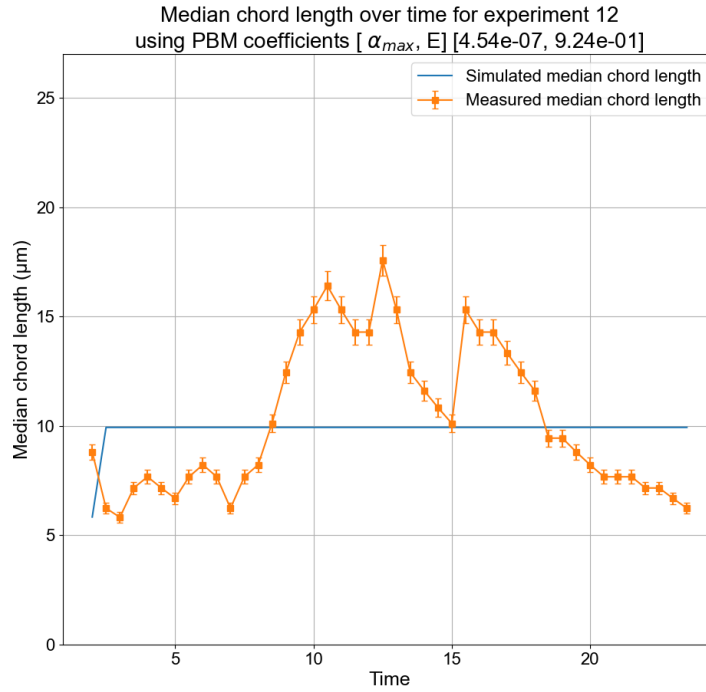


Figure 25: Median of particle size distributions from PBM prediction and experimental data for experiment 12

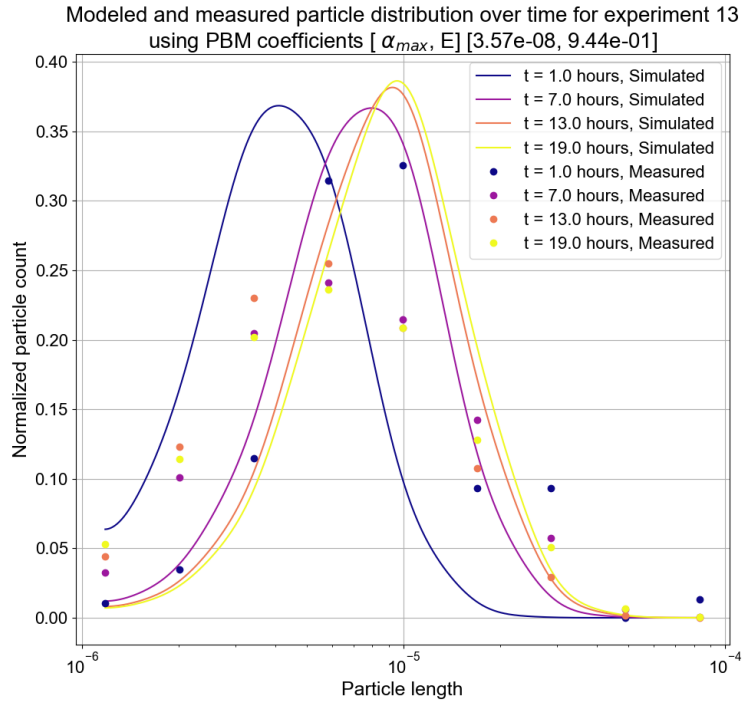


Figure 26: Particle size distributions from PBM model predictions and experimental data for experiment 13

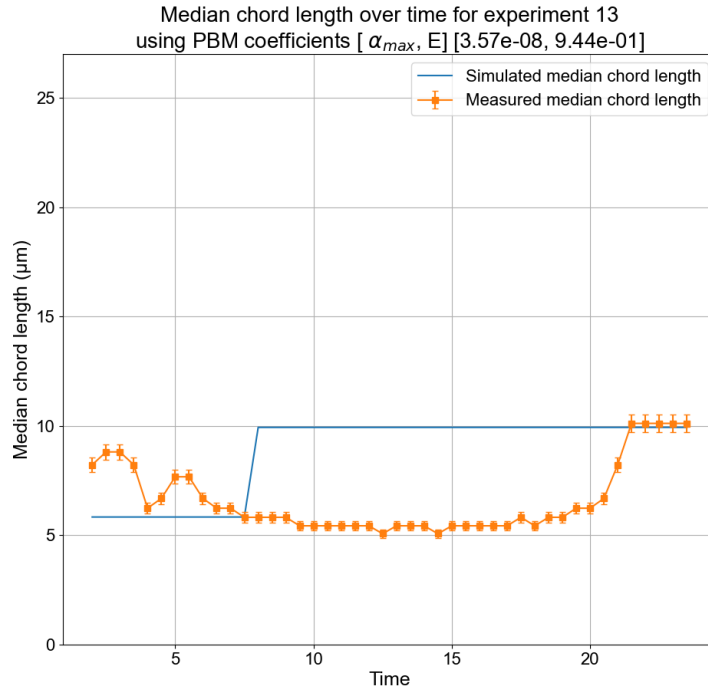


Figure 27: Median of particle size distributions from PBM prediction and experimental data for experiment 13

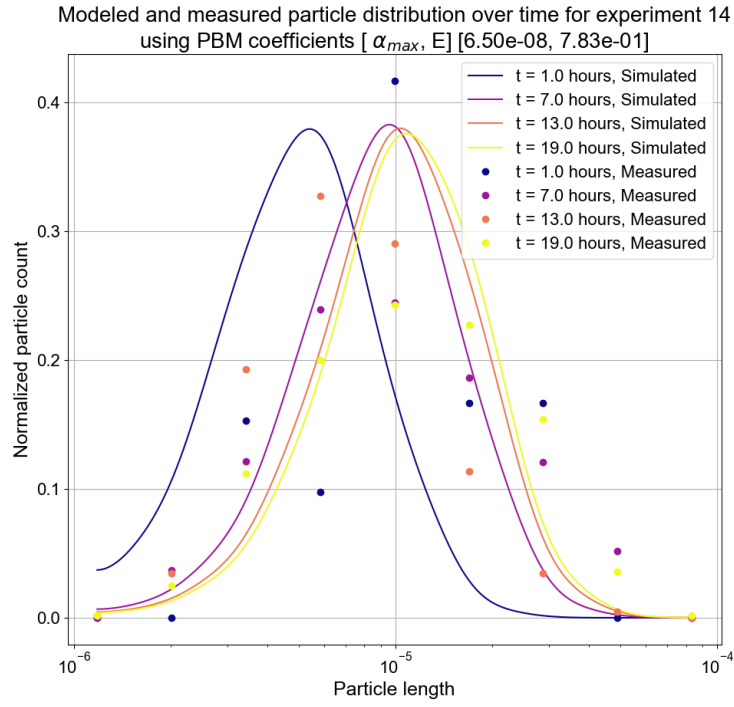


Figure 28: Particle size distributions from PBM model predictions and experimental data for experiment 14

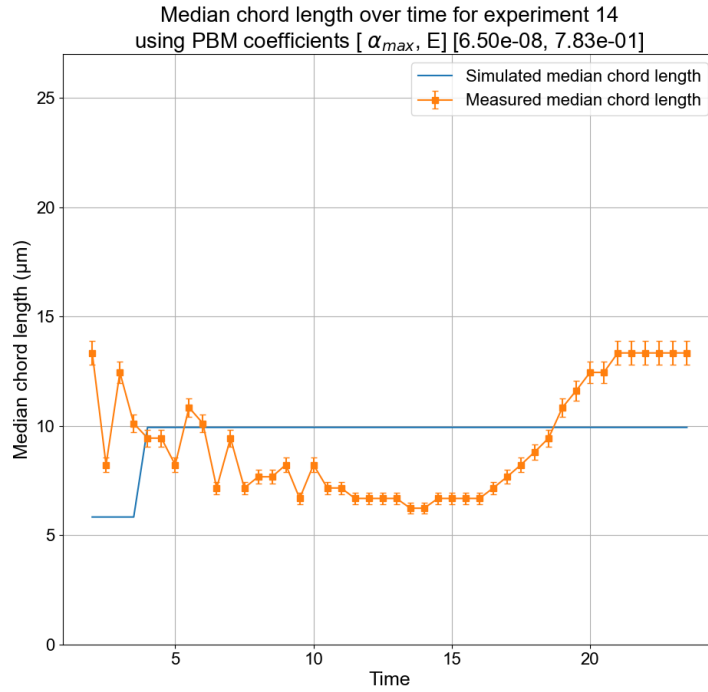


Figure 29: Median of particle size distributions from PBM prediction and experimental data for experiment 14

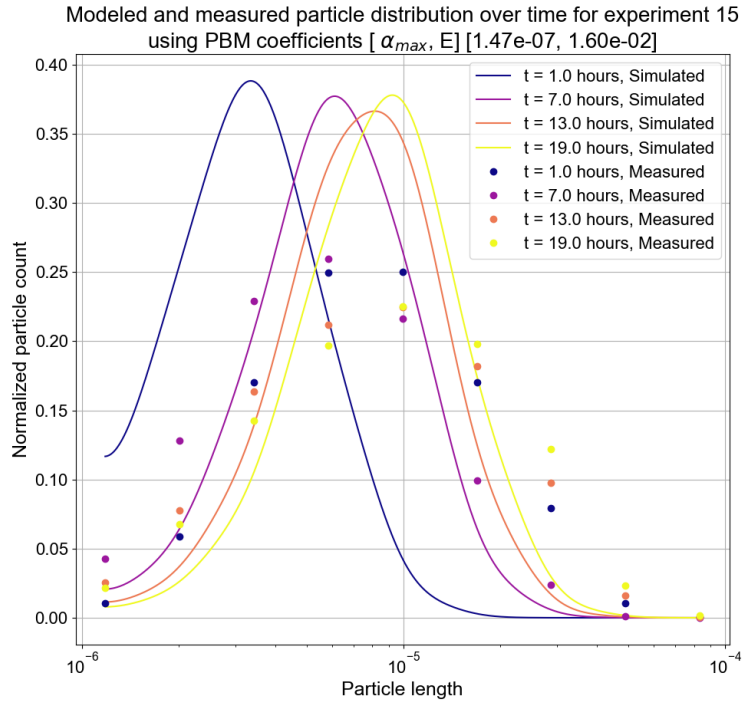


Figure 30: Particle size distributions from PBM model predictions and experimental data for experiment 15

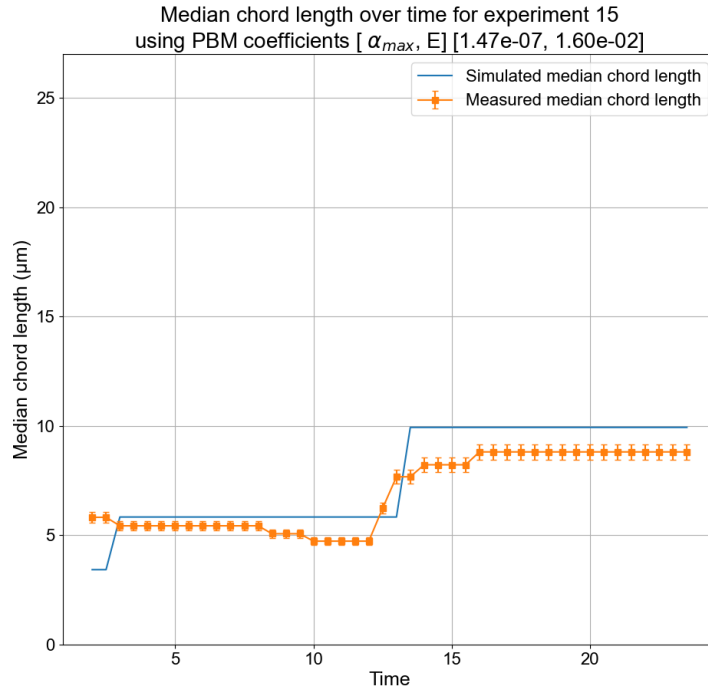


Figure 31: Median of particle size distributions from PBM prediction and experimental data for experiment 15

5 PBM surrogate prediction results:

All of the experimental and surrogate modeling predictions are given in the following section. These are the same experiments but the PBM model is solved with the surrogate predictions.

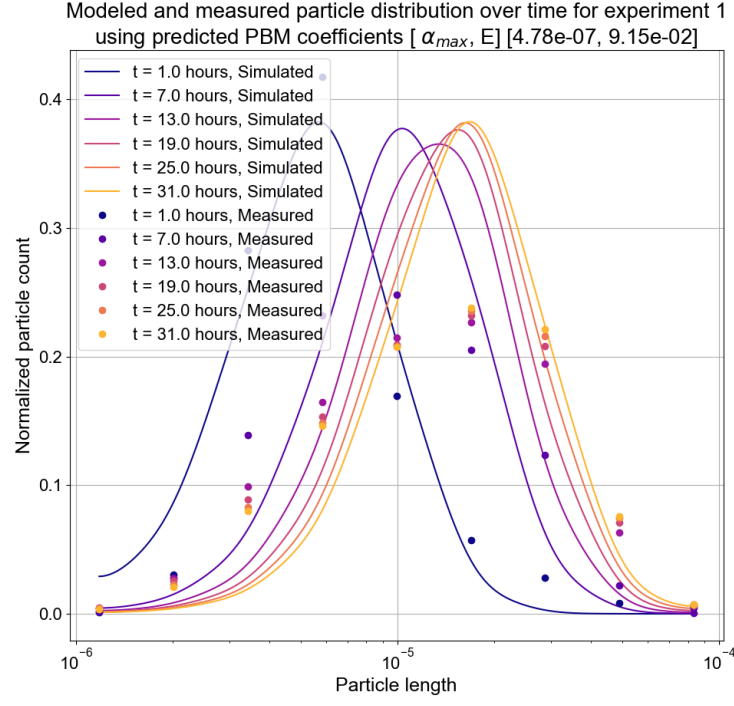


Figure 32: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 1

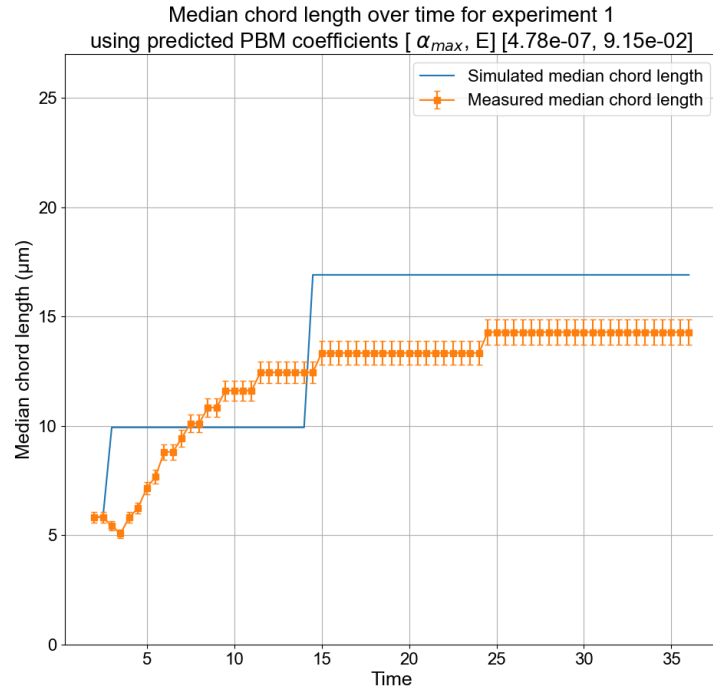


Figure 33: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 1

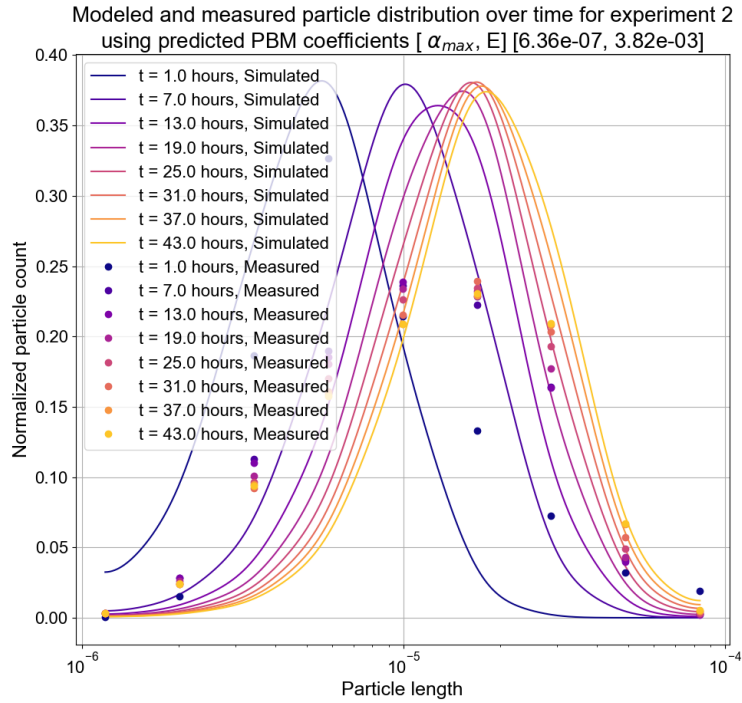


Figure 34: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 2

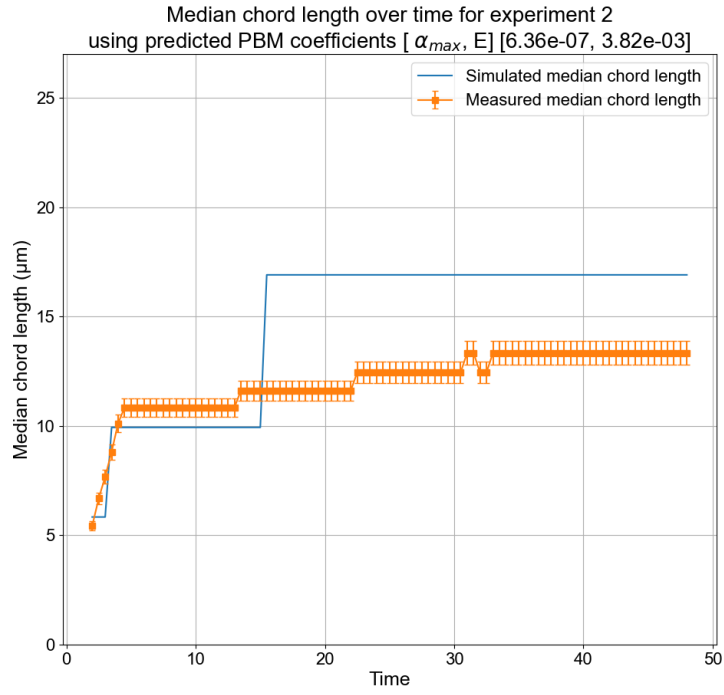


Figure 35: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 2

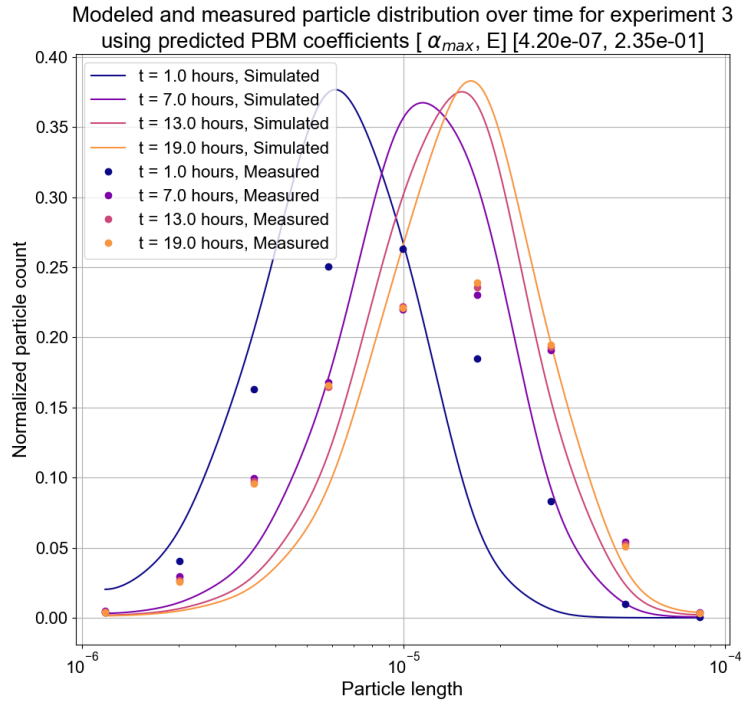


Figure 36: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 3

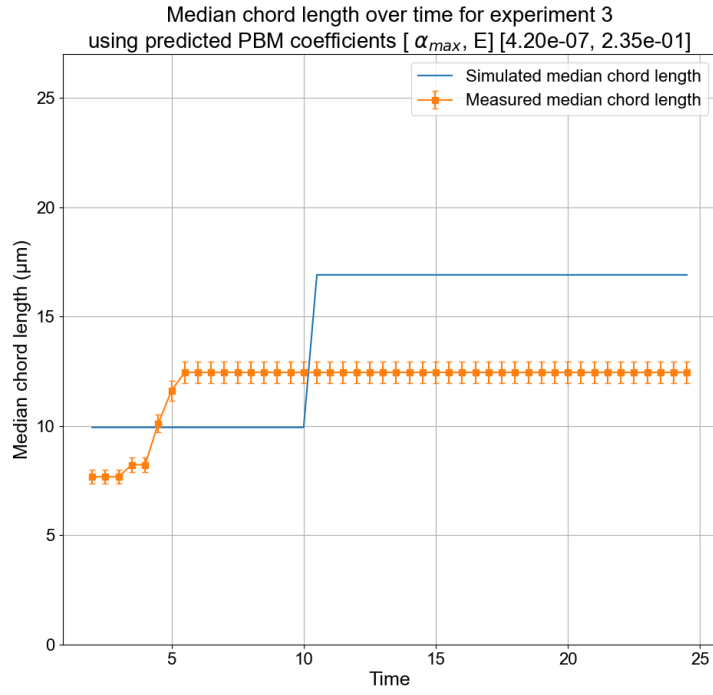


Figure 37: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 3

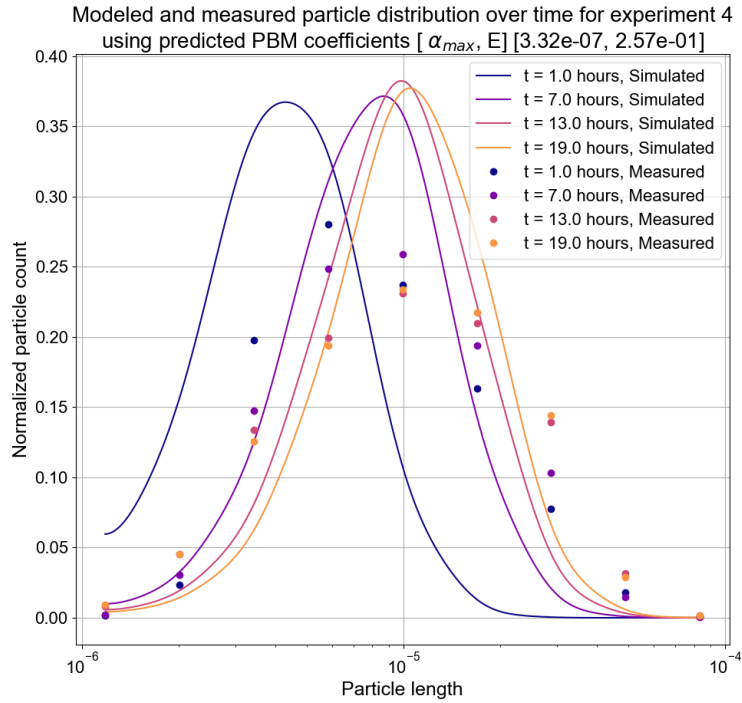


Figure 38: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 4

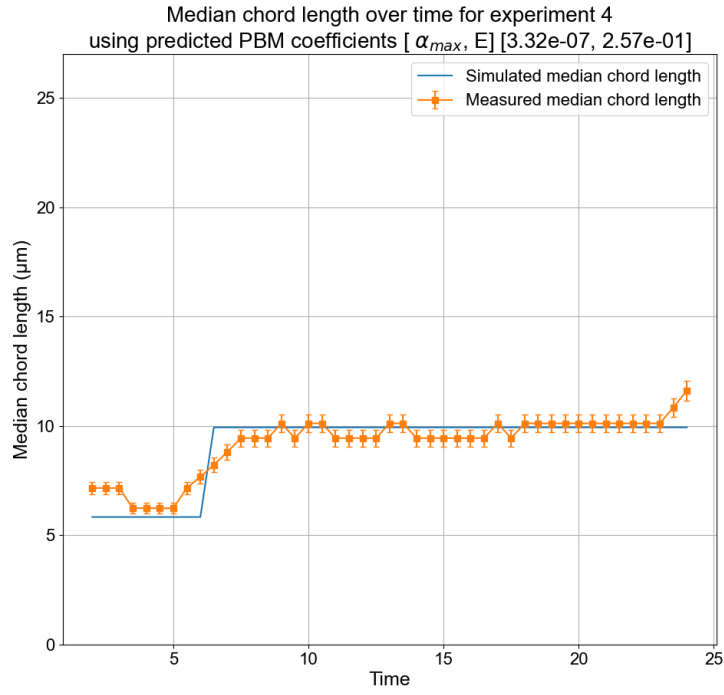


Figure 39: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 4

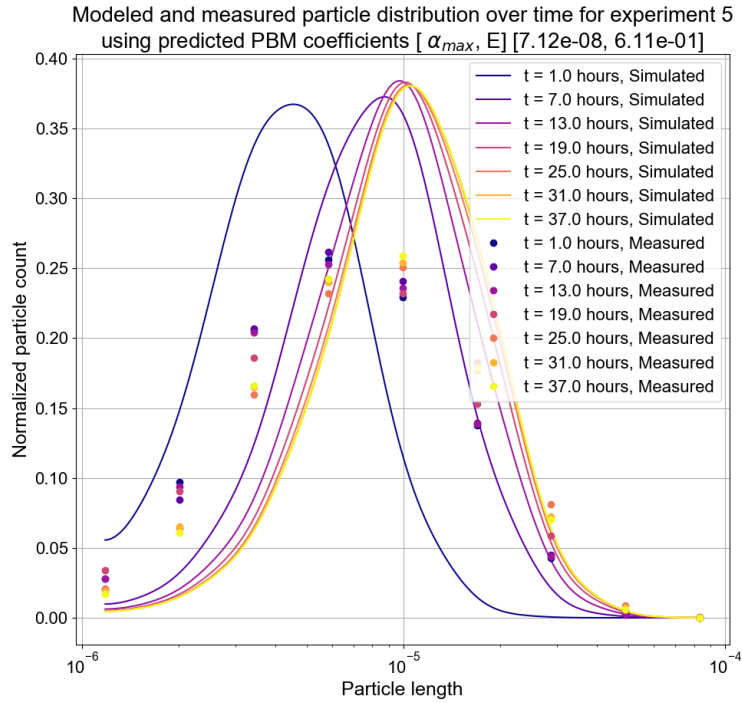


Figure 40: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 5

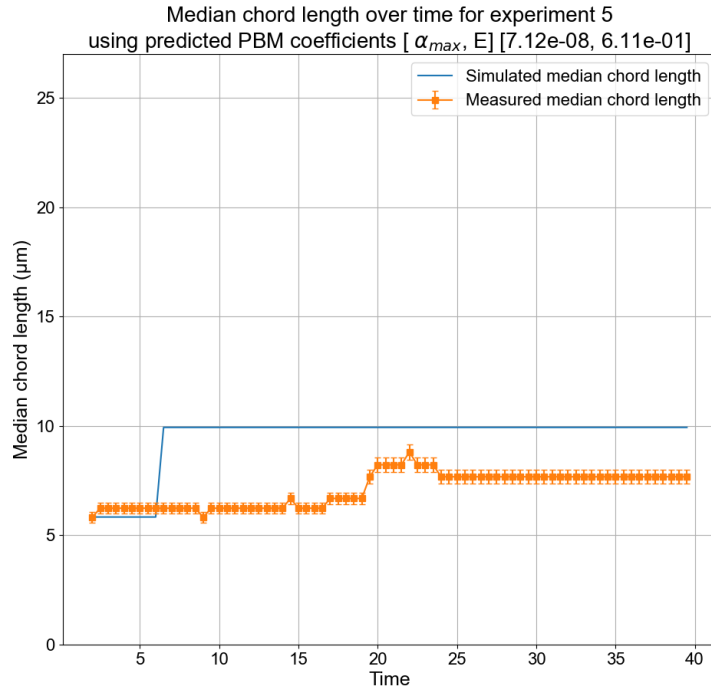


Figure 41: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 5

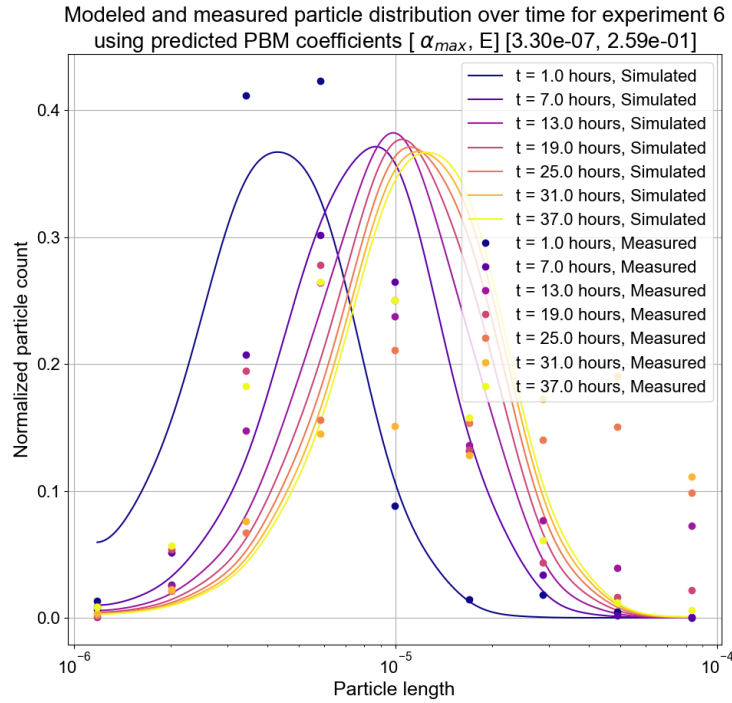


Figure 42: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 6

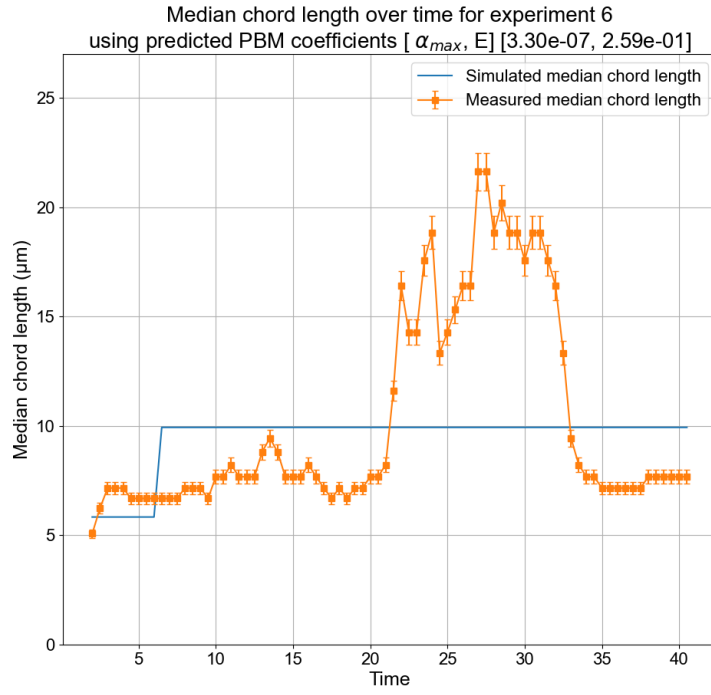


Figure 43: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 6

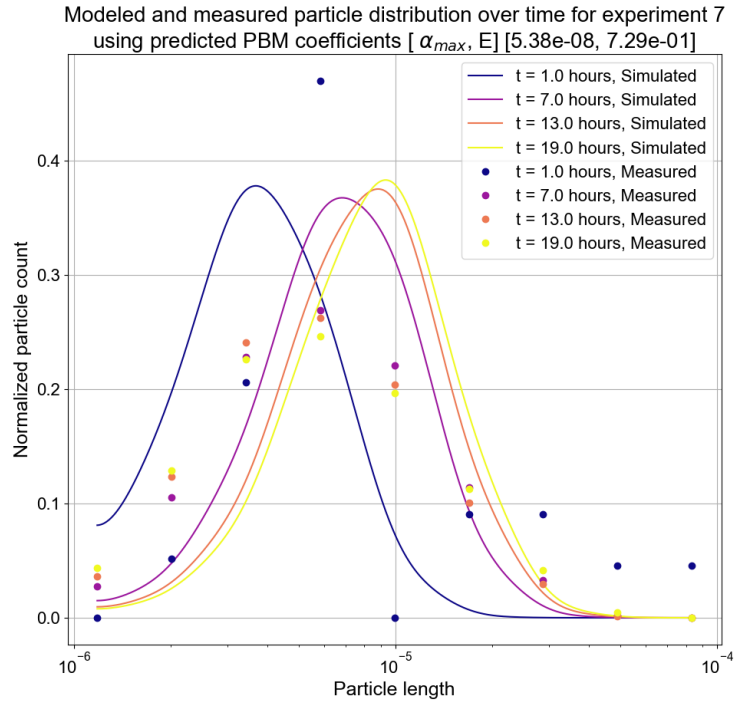


Figure 44: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 7

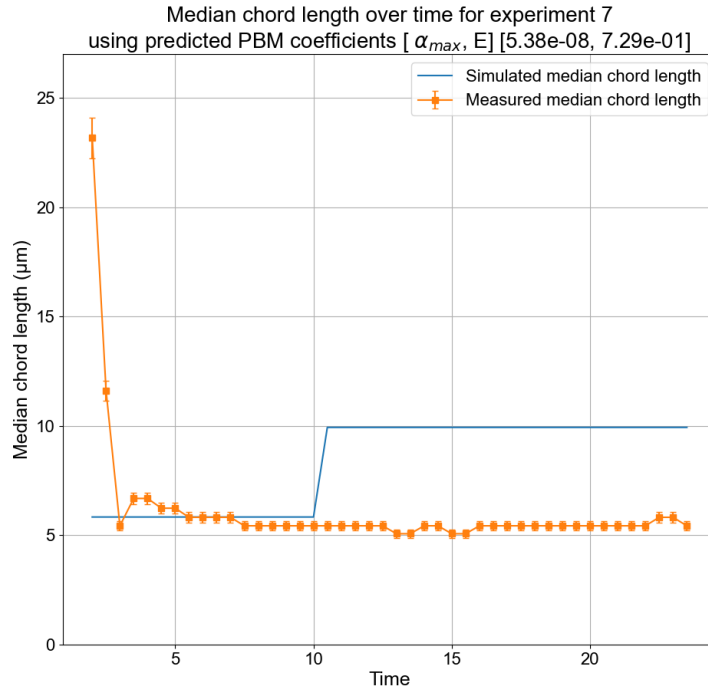


Figure 45: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 7

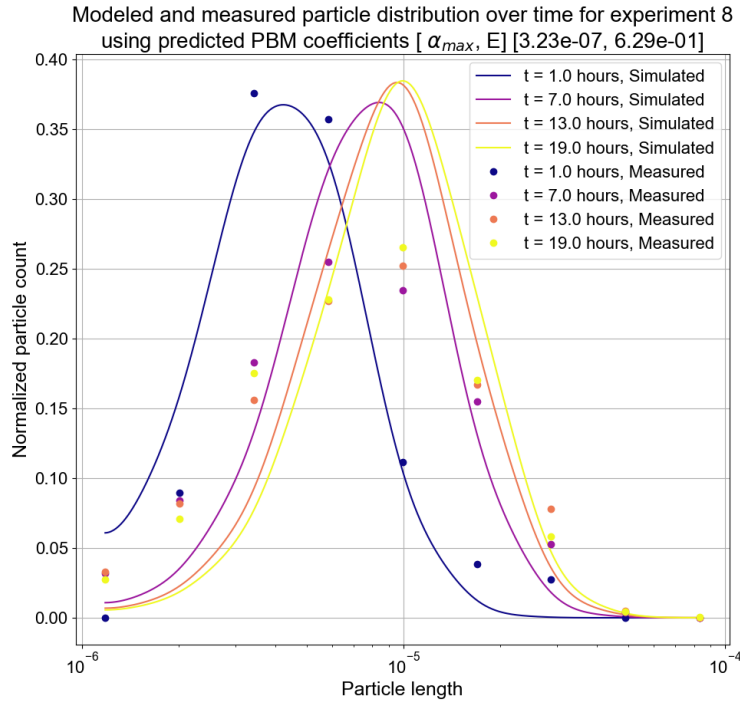


Figure 46: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 8

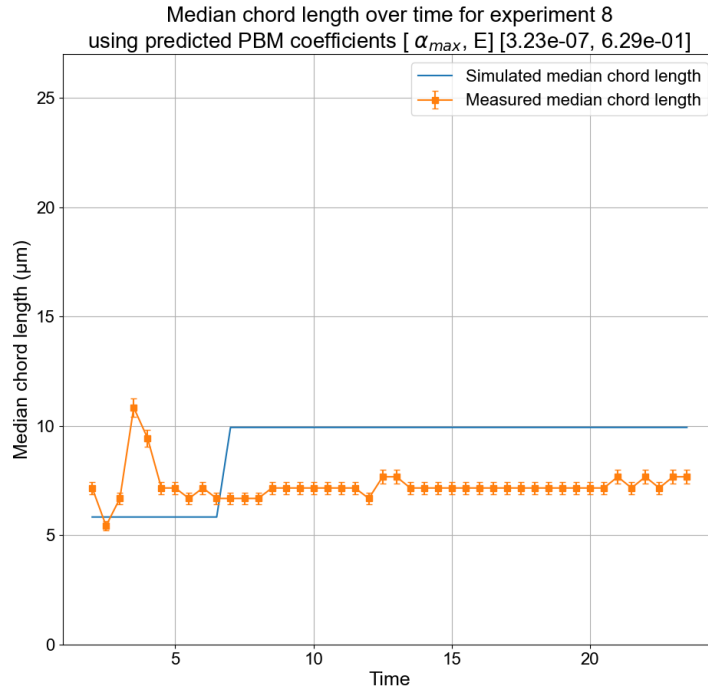


Figure 47: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 8

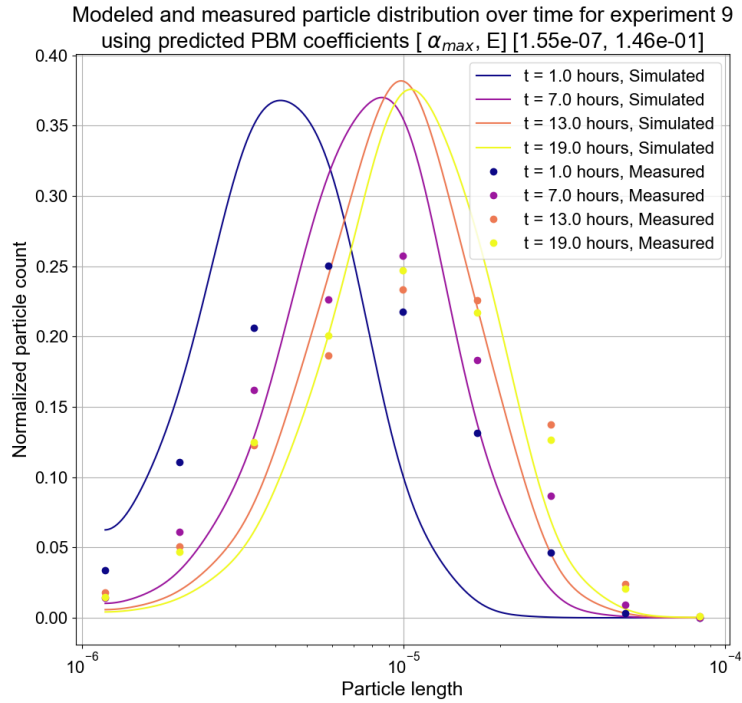


Figure 48: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 9

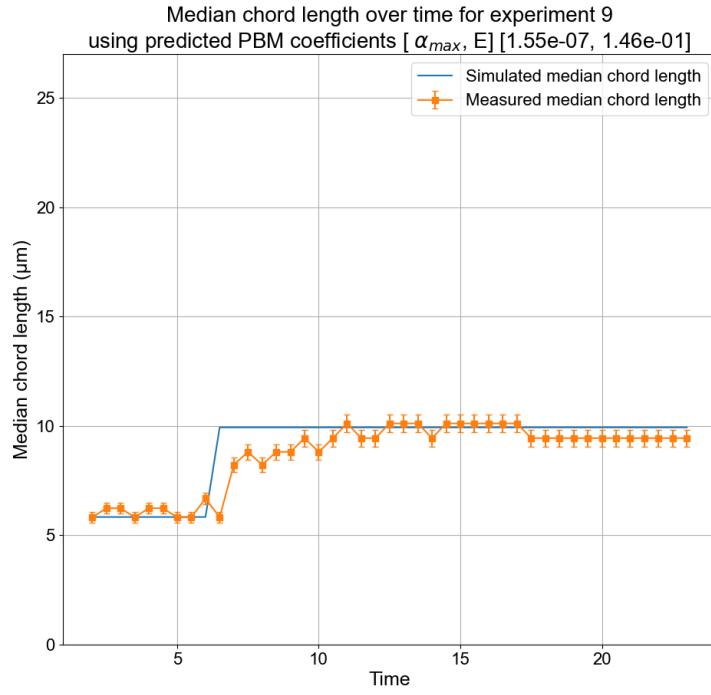


Figure 49: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 9

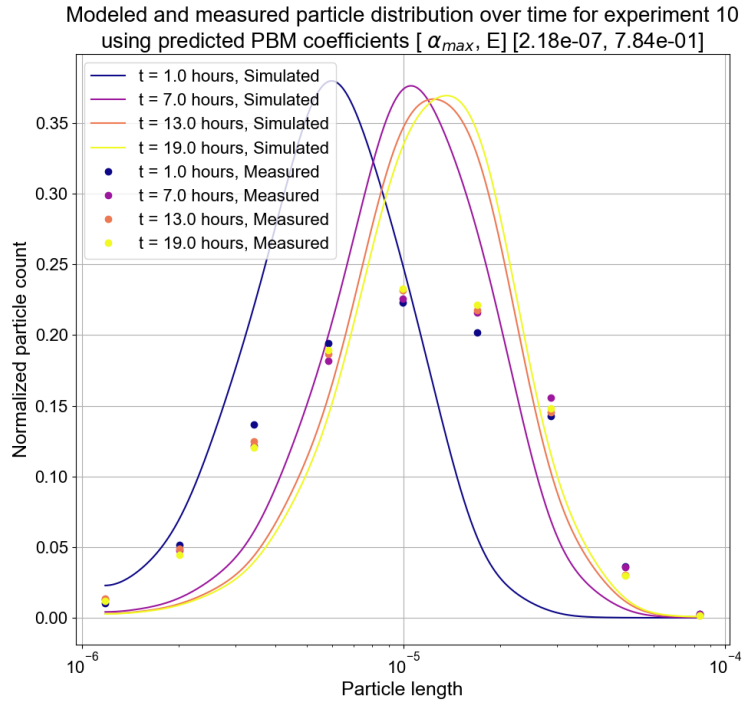


Figure 50: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 10

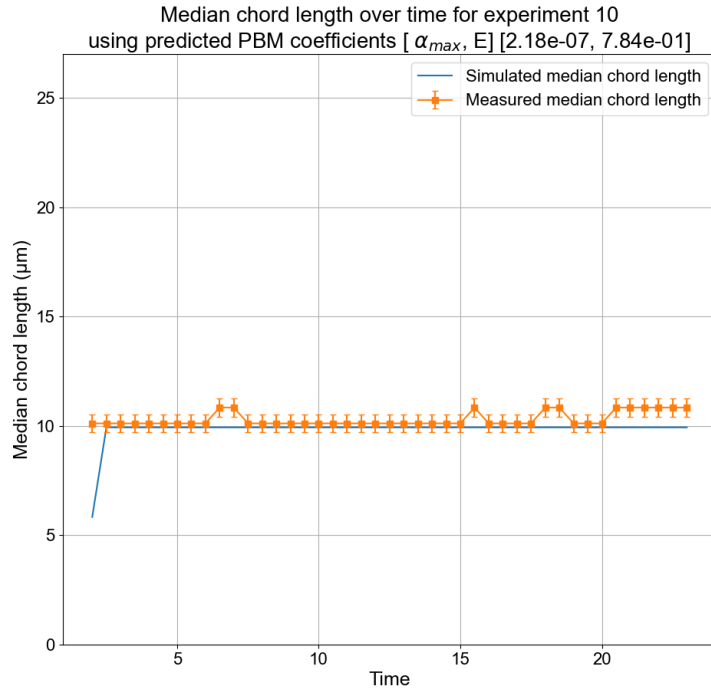


Figure 51: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 10

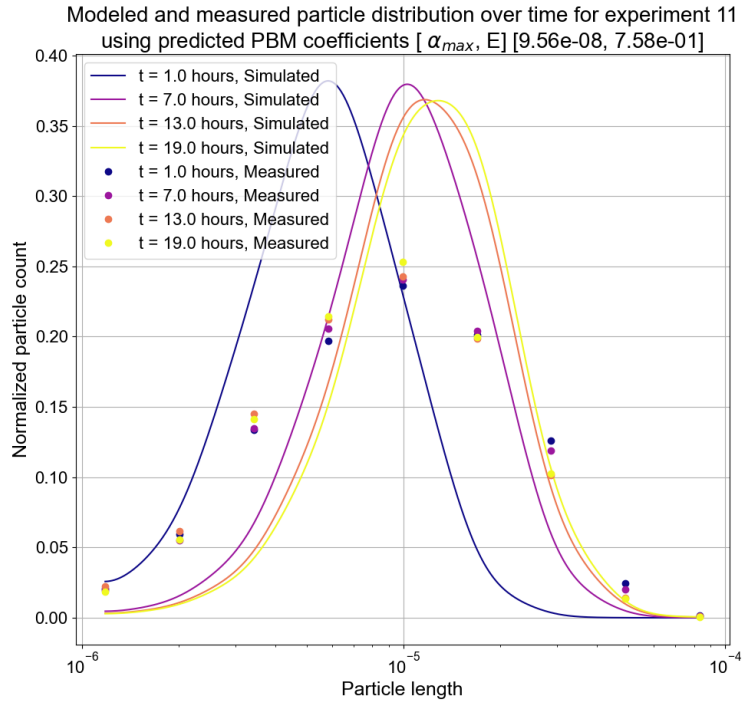


Figure 52: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 11

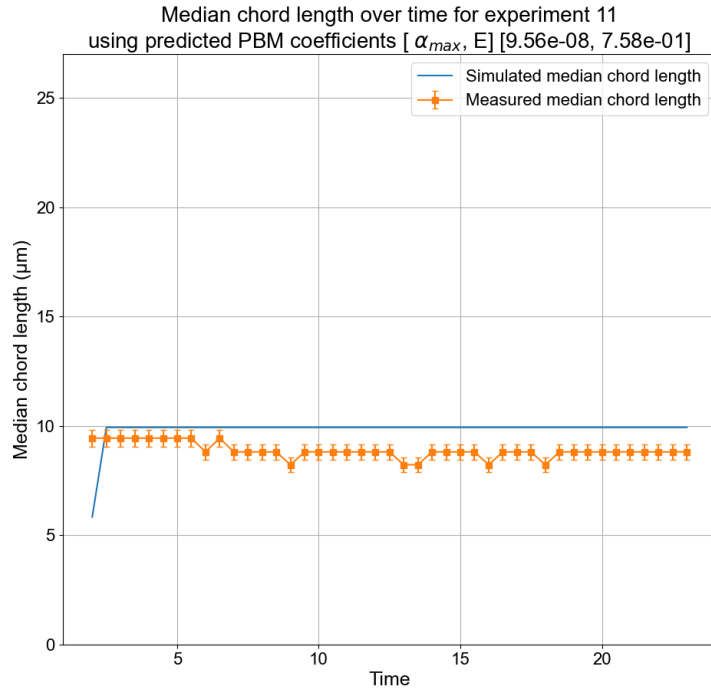


Figure 53: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 11

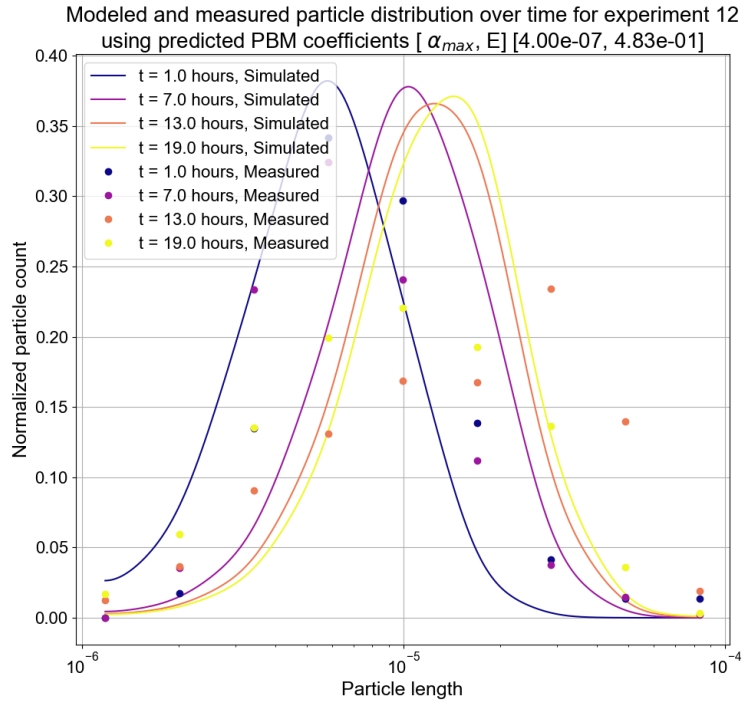


Figure 54: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 12

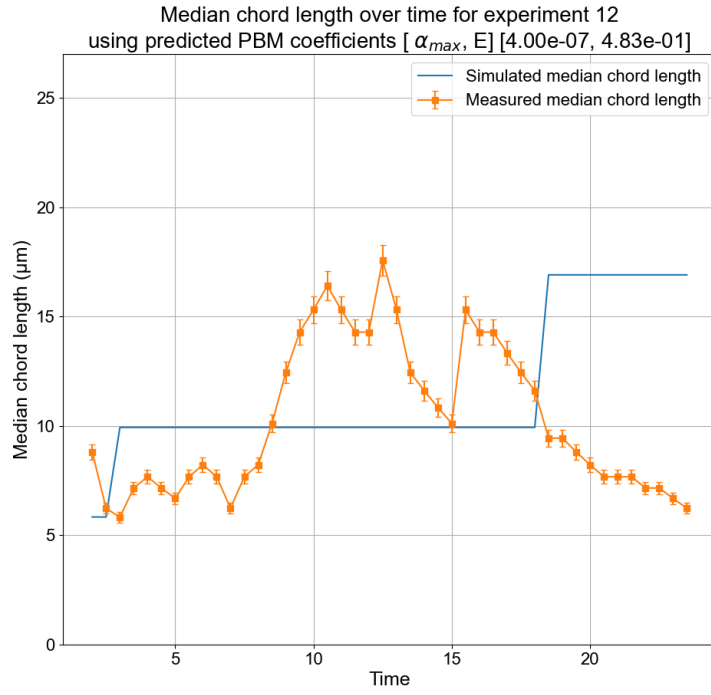


Figure 55: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 12

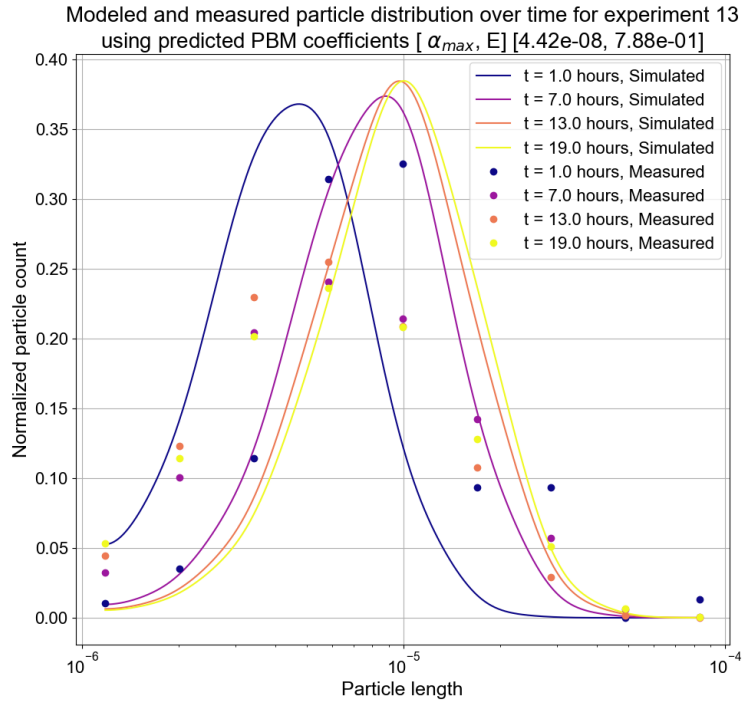


Figure 56: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 13

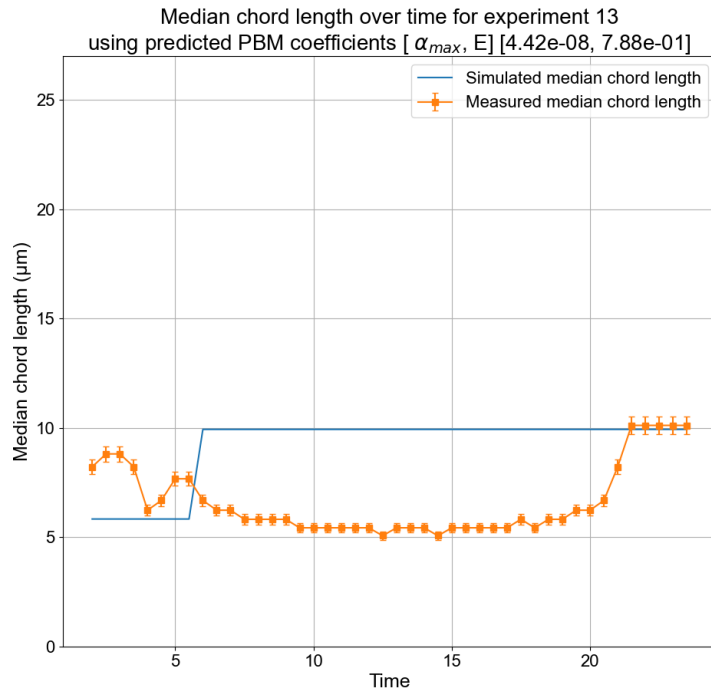


Figure 57: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 13

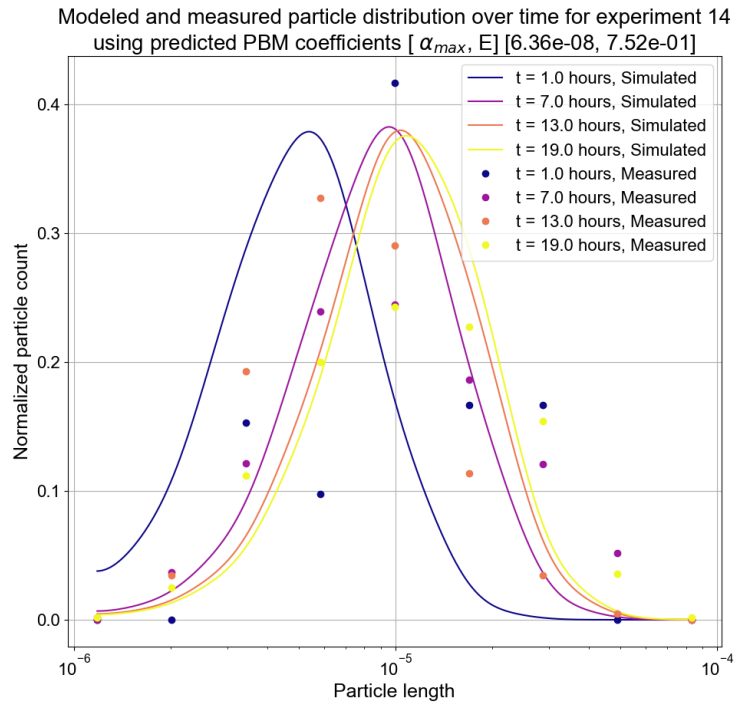


Figure 58: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 14

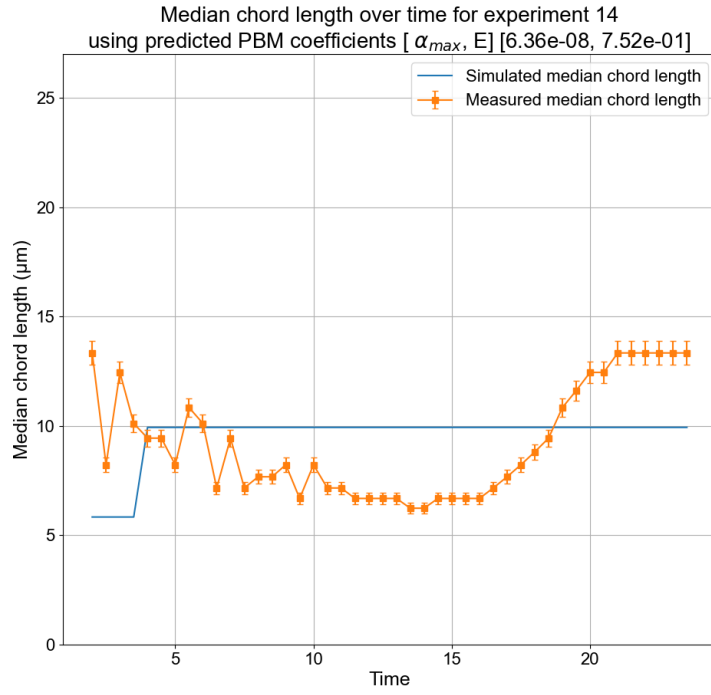


Figure 59: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 14

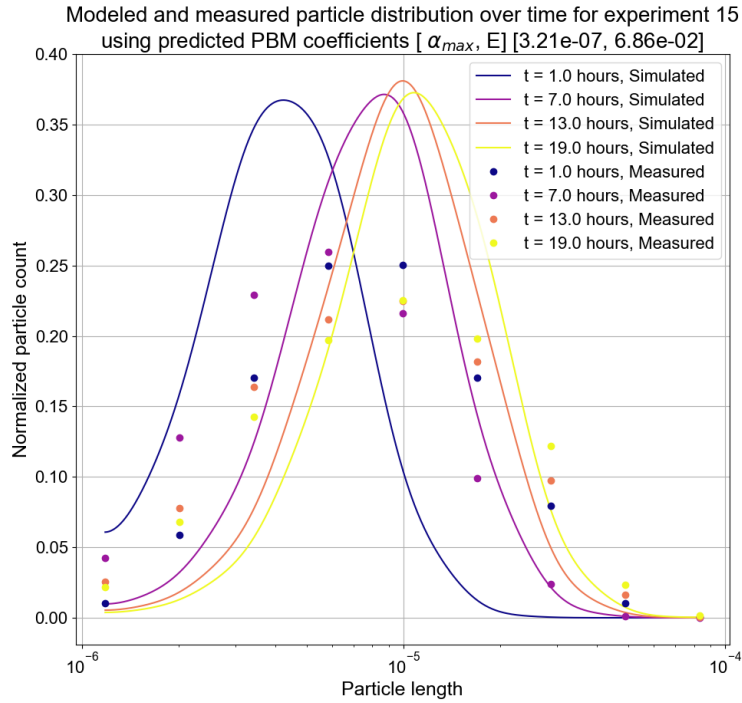


Figure 60: Particle size distributions from ANN-based PBM predictions and experimental data for experiment 15

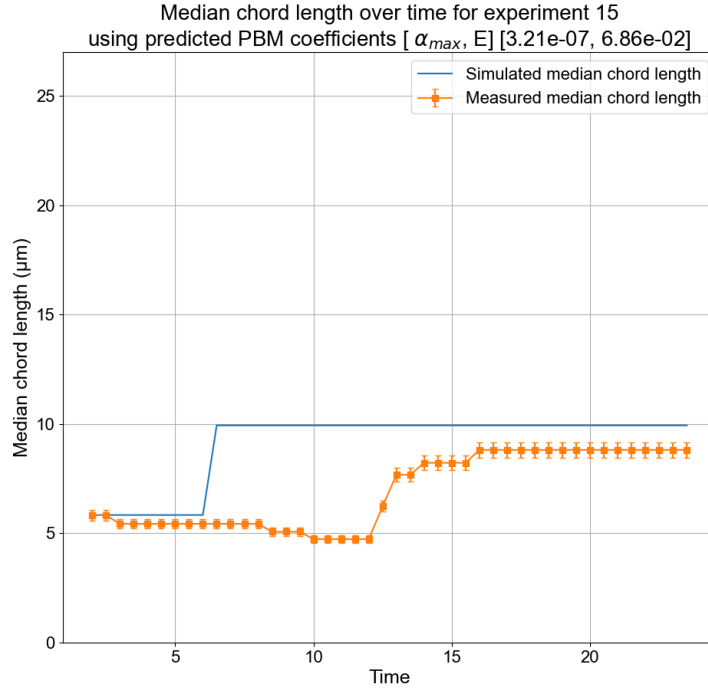


Figure 61: Median of particle size distributions from ANN-based PBM prediction and experimental data for experiment 15

6 Neural network internal variables and structure:

Table 4: Structure of Feedforward Artificial Neural Network (FFANN)

Layer	Type	Input Size	Output Size	Additional Info
1	Linear	4	64	First hidden layer
dropout 1	Dropout	64	64	Dropout with $p = 0.1$
2	Linear	64	32	Second hidden layer
dropout 2	Dropout	32	32	Dropout with $p = 0.1$
3	Linear	32	2	Output layer
relu	Activation	-	-	ReLU activation after each layer

The table above describes the structure of the Feedforward Artificial Neural Network (FFANN) used in this study. Each layer of the network serves a specific purpose in processing the data. Below is a detailed explanation of the table:

- **Layer 1 (Linear):** This is the first hidden layer of the network. It takes 4 input features and transforms them into 64 output features. This layer learns initial patterns from the data.
- **Dropout 1:** Dropout is a regularization technique used to prevent overfitting by randomly deactivating some neurons during training. In this case, a dropout rate of $p = 0.1$ means 10% of the neurons in this layer are deactivated during each training iteration.

- **Layer 2 (Linear):** This is the second hidden layer, which takes the 64 outputs from the previous layer and reduces them to 32 outputs. This layer helps in learning more complex patterns in the data.
- **Dropout 2:** Similar to Dropout 1, this layer applies a dropout rate of $p = 0.1$ to the 32 neurons to further regularize the network and improve its generalization capability.
- **Layer 3 (Linear):** This is the output layer of the network. It takes 32 inputs and reduces them to 2 outputs, which represent the final predictions of the model.
- **ReLU Activation:** The ReLU (Rectified Linear Unit) activation function is applied after each linear layer. ReLU is widely used due to its simplicity and effectiveness in introducing non-linearity, making it easier for the network to model complex relationships in the data.

The neural network was implemented using the PyTorch library. The design includes two hidden layers, which are generally sufficient for learning patterns in most datasets. The inclusion of dropout layers adds regularization, helping to prevent overfitting, while the use of the ReLU activation function ensures the network is versatile and capable of handling diverse data types.

7 Neural network training and validation

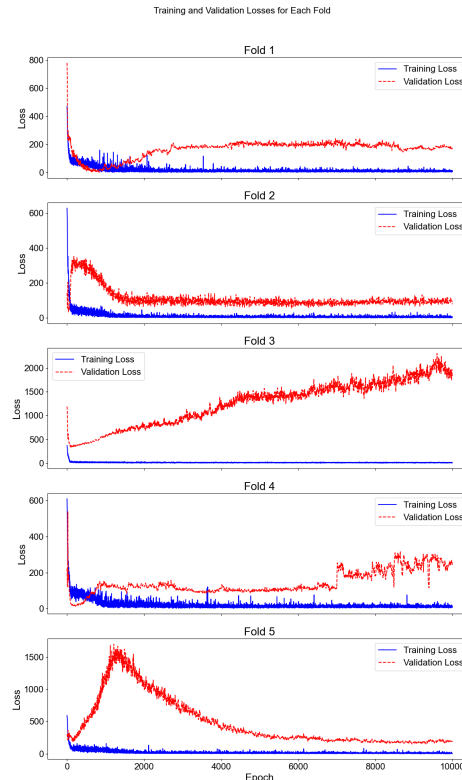


Figure 62: Training and validation loss of the ANN over epochs on every fold. The model is showing signs of overfitting in 3 middle folds.

The model has been trained and the outputs are averaged over 5 folds and 10000 epochs each. The training and validation loss has been presented in Fig. 62.

In Fig. 62 In two of the folds, the model performs well, with both validation and training errors following similar trends, indicating consistent learning. However, in other folds, the validation loss deviates from the training loss, suggesting signs of overfitting. Despite this, averaging the predictions across folds helps regularize the model, reducing the impact of overfitting on the final results. The number of epochs was set to 10000 to ensure the model had sufficient time to learn the data.

References

- [1] N. Nazemzadeh, An integrated multi-scale modeling framework for flocculation processes, Phd dissertation, Technical University of Denmark (2022).
- [2] R. E. CANFIELD, Peptides Derived from Tryptic Digestion of Egg White Lysozyme, *Journal of Biological Chemistry* 238 (8) (1963) 2691–2697. doi:10.1016/S0021-9258(18)67887-1.